UNIT IV

Security – Security Basics – Security Issues – Types of Security Attacks – WS –Security.
Mobile and Wireless – Mobile Web Services – Challenges with mobile – Proxy Based Mobile Systems -Direct Mobile Web service access - J2ME Web Services.Building Real World Enterprise Web Service and Applications: Real World Web Service Application Development – Development of Web services and Applications onto Tomcat application Server and Axis Soap Server.

SECURITY

The most critical issue limiting the widespread deployment of Web services by organizations is the lack of understanding of the security risks involved as well as the best practices for addressing those risks. Development and IT managers want to know whether the security risks and the types of attack common for Web sites will be the same for Web services. Will existing enterprise security infrastructure already in place, such as firewalls and well-understood technologies like Secure Sockets Layer (SSL), be sufficient to protect their companies from Web service security risks?

Much of the experience companies have with security is with Web sites and Web applications. Both Web sites and Web applications involve sending HTML between the server and the client (e.g., Internet browser). Web services involve applications exchanging data and information through defined application programming interfaces (APIs). These APIs can contain literally dozens of methods and operations, each of which presents hackers with potential

Security Is An End-to-End Process

Contrary to popular belief, security is important not only during data transport between one computer and another computer, but also after transport. In the process of doing a transaction the path that data follows is oftentimes complex and long, involving multiple hops. If a small section of this path is insecure, the security of the entire transaction and the system is compromised.

Today, many systems that are seemingly secure are in fact insecure. Designers and architects usually focus on the security issues relating to the transmission of data between the client and the server, while other segments of the data transmission value chain are assumed to be secure and do not get much attention. Many of today's Web site and Web application vulnerabilities are directly applicable to Web services as these and other legacy systems are oftentimes just being wrapped and made available as Web services.

Data Protection and Encryption

Data protection refers to the management of transmitted messages so that the contents of each message arrives at its destination intact, unaltered, and not viewed by anyone along the way. The concept of data protection is made up of the sub-concepts of data integrity and data privacy:

TYPES OF SECURITY ATTACKS AND THREATS

Types of Security Attacks and Threats

Since Web services leverage much of the infrastructure developed for Web sites, it is understandable that the types of security breaches that are common for Web sites will also be common for Web services.

However, since Web services provide an application programming interface (API) for external agents to interact with it and also provides a description of this API (in the form of WSDL files), Web service environments facilitate and in fact attract attacks. These environments also make it more difficult to detect attacks from legitimate interactions.

Malicious Attacks

As we have discussed, Web service traffic shares a lot in common with Web site traffic.

Both types of traffic usually flow through the same ports and while many firewalls can recognize SOAP traffic, they usually treat them as standard HTTP traffic.

1. Denial of Service (DoS)

Definition

A **Denial of Service (DoS)** attack is an attempt to make a computer system, server, website, or network resource unavailable to legitimate users by overwhelming it with excessive traffic or malicious requests.

The primary objective is to **disrupt service availability**.

Characteristics

- Single attacking machine.
- Targets one victim system.
- Consumes network bandwidth, CPU, memory, or other resources.
- Prevents legitimate users from accessing services.

DoS by Flooding

In flooding attacks, the attacker sends a huge volume of packets or requests to the target.

Working

1. Attacker generates excessive traffic.
2. Network resources become congested.
3. Server resources get exhausted.
4. Legitimate requests are dropped or delayed.

Examples

a) DNS Flooding

- Large numbers of DNS requests are sent.
- DNS server becomes overloaded.
- Legitimate DNS queries cannot be processed.

b) ARP Flooding (Address Resolution Protocol)

- Attacker sends many fake ARP messages.
- Network switches become overloaded.
- Causes network instability.

c) Ping Broadcast Attack

- ICMP(Internet Control Message Protocol) Echo Requests are sent to a broadcast address.
- Multiple systems reply simultaneously.
- Victim receives an overwhelming amount of traffic.

d) TCP SYN Flooding

One of the most common DoS attacks.

normal TCP Three-Way Handshake

1. Client → SYN
2. Server → SYN-ACK
3. Client → ACK

Connection established.

SYN Flood Attack

1. Attacker sends many SYN packets.
2. Server allocates resources for each request.
3. Attacker never sends the final ACK.
4. Server waits for completion.
5. Connection queue becomes full.

Result:

- New legitimate users cannot connect.

DoS by Forging

Definition

The attacker sends fake or forged routing information.

Working

- False routing updates are transmitted.
- Routers receive incorrect path information.
- Packets are redirected or dropped.
- Communication is disrupted.

Impact

- Network congestion.
- Misrouting of packets.
- Service interruption.

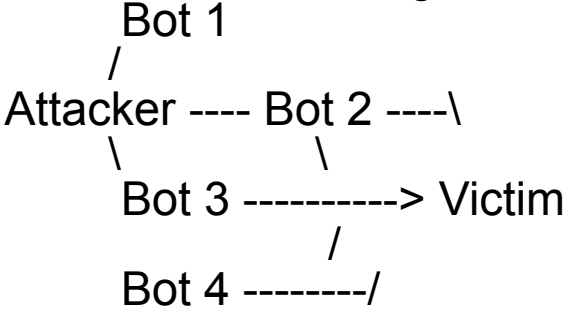
2. Distributed Denial of Service (DDoS)

A **Distributed Denial of Service (DDoS)** attack is an advanced form of DoS in which multiple compromised computers attack a target simultaneously.

Instead of one attacker, thousands or millions of systems generate traffic.

Working of DDoS Attack

As shown in the diagram:



Steps

1. Attacker infects many computers with malware.
2. These compromised computers become **bots** or **zombies**.
3. Collection of bots forms a **Botnet**.
4. Attacker sends commands to the botnet.
5. All bots simultaneously attack the victim.
6. Victim becomes overloaded and unavailable.

Characteristics of DDoS

- Multiple attack sources.
- Difficult to identify real attacker.
- Extremely high traffic volume.
- Harder to mitigate than DoS.
- Often uses globally distributed botnets.

Examples of DDoS Attacks

HTTP Flood

- Large number of web page requests.
- Web server resources exhausted.

DNS Amplification

- Small DNS query generates a large response.
- Traffic is amplified toward victim.

UDP Flood

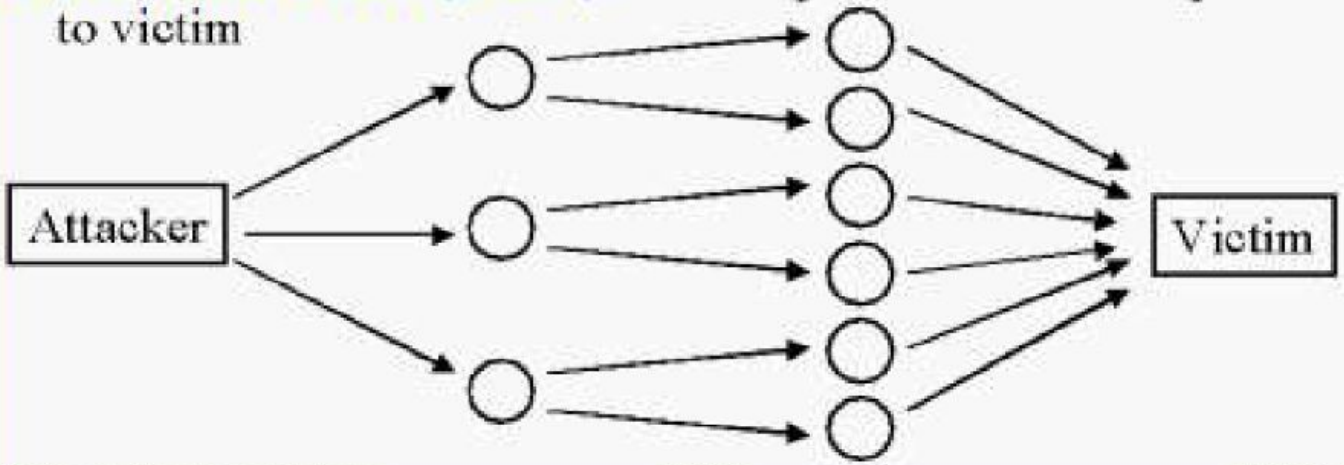
- Massive UDP packets sent.
- Bandwidth gets consumed.

ICMP Flood

- Huge number of ping requests.
- Network resources become exhausted.

Types of Security Attacks

- **Denial of Service (DoS)**
  - DoS by Flooding: Lots of packets from one node to victim.
    DoS on DNS or root name servers.
    ARP flooding, ping broadcasts, TCP SYN flooding.
  - DoS by Forging: Send incorrect routing message
- **Distributed DoS (DDoS):** Lots of packets from multiple nodes to victim



Security Attacks (Cont)

- **Trojan Horse**: Programs with hidden functionality. Could be triggered when a specific time or condition.
- **Trap Doors**: Backdoor. Code segment to circumvent access control.
- **Virus**: A program that reproduces by introducing a copy of itself in other programs. Jump to Viral code and return to beginning.
- **Worms**: Creates copies of itself on other machines.
  Unlike virus, worms do not require user action.
  Morris worm spread by finding IP addresses on the machine.
  Slammer worm sent UDP packets to cause buffer overflow.
- **Buffer Overflow**: Overwrite code segments and execute code in data space. Many programming languages do enforce bound checking.

Security Attacks (Cont)

- **Covert Communications Channel**: Hidden channel.
  - Capture electromagnetic radiations from keyboards, screens, and processors.
  - Pizza deliveries to White House
- **Steganography** or Information Hiding: Lower bits of pictures or music files.
- **Reverse Engineering**: dismantling and inspecting to infer internal function and structure. Code dumping and decompiling
- **Scavenging**: Acquisition of data from residue. Searching through rubbish bins. Buffer space in memory, deleted files on disks, bad blocks on disks
- **Cryptanalysis**: Find encryption key, encryption method, or clear text. Get plain-text and cipher text pairs.

Security Solutions

- **Audits**: May including testing by a red team.
  Keep good system logs.
- **Formal methods**: Used to verify no human errors in the code and protocols.
- **Attack Graphs**: Show paths that an attacker can take to get access
- **Security Automata**: Security policies expressed as finite state machines.
- **Encryption**: Secret key and public key
- **Steganography**: Digital water marking. Information hidden in images, sound, or video can be used to find the origin of data.

WS-SECURITY

WS-Security is a specification that unifies multiple Web services security technologies and models in an effort to support interoperability between systems in a language- and platform-independent manner. More specifically, WS-Security specifies a set of SOAP

extensions that can be used to implement message integrity and confidentiality. To this end, the WS-Security specification brings together a number of XML security technologies and positions them within the context of SOAP messages.

The origins of WS-Security are with Microsoft, IBM, and VeriSign submitting a group of security specifications to the Organization for the Advancement of Structured Information Standards (OASIS). Later, Sun Microsystems started to cooperate to further develop the specifications. With such heavyweights behind it, WS-Security is emerging as the de facto standard for Web services security.

he lack of a coherent security model and policy is the most often cited reason for the slow deployment of externally facing Web services by enterprises. Addressing security issues with vigor will not only make Web services (as well as the applications that consume Web services) more secure, but will also increase the community's confidence in Web service technologies. This improved confidence will likely result in increased numbers and types of available Web services.

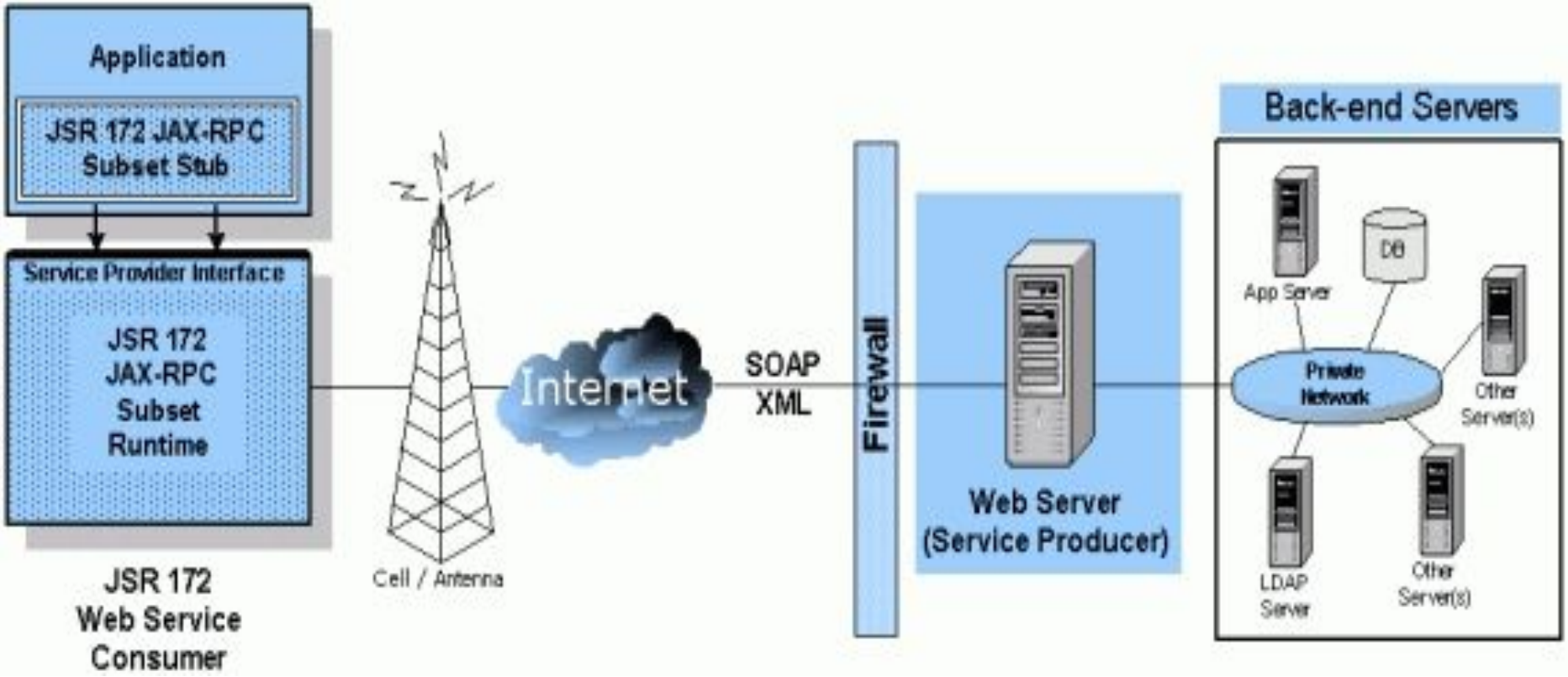
Mobile and Wireless – Mobile Web Services

What is a Mobile Agent?

This is a simpler question to answer - at it's most basic level we can say that an agent is mobile if we don't need to know where in the system it resides. Thus, a mobile agent could

be found on a desktop computer, a mobile device such as a PDA, or even in the depths of a server. All we need to know is how to find the agent, and how to communicate with it.

Once we have this conceptual framework in place, we have the abstraction and flexibility required to design our system. How mobile is a mobile agent, though? Currently, in order to move from one system to another, or even to communicate amongst themselves, mobile agents need a common platform on which to operate. Thus, in order for our mobile agents to be useful to our business partners (and to ourselves by operating with our partners), we have to share a platform with our partners, something we cannot guarantee being able to do. Even with the best developers in the world, this will take time. In order for intelligent agents to communicate, they would need to share a common language - the best candidate for this would be XML, or SOAP.


Application
JSR 172 JAX-RPC Subset Stub
Service Provider Interface
JSR 172 JAX-RPC Subset Runtime
JSR 172 Web Service Consumer
Cell / Antenna
Internet
SOAP XML
Firewall
Web Server (Service Producer)
Back-end Servers
App Server
DB
Private Network
Other Server(s)
LDAP Server
Other Server(s)

Mobile Agents and Web Services

With the advent of Web Services, we can develop mobile agents that can exist on an Intranet, or even in the deeper waters of the Internet. How? Because exposing our mobile agents as Web Services means that we can take advantage of the already existing network infrastructure and communication protocols provided by the Internet. As we expose our existing mobile agents as Web Services, or write future agents to take full advantage of the situation, not only do we make it easy for ourselves, but also quicker and more convenient.

Mobile intelligent agents can be developed to the Web Services standard using SOAP, WSDL, and UDDI. The mobile intelligent agents are Web Services ready so they can be plugged into this network and serve its knowledge. Using WSDL give the agent the ability to describe it's capacity, invaluable if it is to become part of a domain agent society.

As if the ability to use the Internet to further the reach of our mobile agents wasn't enough, there are companies dedicated to providing networking facilities for Web Services. With Web Services essentially moving components onto the Internet, it makes sense for companies to provide technology that will ease this process. Using such facilities, we can launch our mobile agents into the wider world should we desire. Companies such as Grand Central and Flamenco Networks are providing these Web Services network solutions.

Advantages of using Web Services

As well as making it easy for our mobile agents to be truly mobile, Web Services can provide other important benefits. The following list represents the key additional benefits:

1. Conservation of Enterprise bandwidth
2. Reduction in latency
3. Conflict Knowledge Credibility Management
4. Support for Dynamic Deployment
5. Improved decision support workflow

Enterprise bandwidth is conserved through the use of mobile agents due to the reduction in network traffic they enable. Rather than pass numerous requests across the network, we can send a mobile agent to the site requiring processing; once there, the agent can conduct the operation, and return to the server only the information that actually needs to be returned. Of course, there will be a point when it isn't efficient to use a mobile agent, like when the process involves only two or three network calls.

Our mobile agents would ideally reside on a Web Services network. If a user requests knowledge from a mobile agent very frequently, the Web Services network can intelligently save this knowledge into a cache. So when a user asks the same questions, the Web Service cache will broadcast it for mobile agent if there is nothing new in the request.

A Web Services network will host many mobile agents, each of which will have expert knowledge in a particular area. The Web Services network will also need intelligence to be able to tell which agent has the best knowledge or if the knowledge conflicts with that of another agent. The Web Services network will judge and pick the best by its own intelligence and broadcast to the requesting agent.

As you would expect, mobile agents can aid in the dynamic deployment of new components in a Web Service. Additionally, a mobile agent can act as a remote controller in a process, allowing decision support workflow to be improved.

With our mobile agents residing in a Web Services network, many of our infrastructure concerns are handled for us. When a user requests certain knowledge, the Web Service network will route this message to a mobile agent. The network determines which is the nearest agent that has the knowledge requested, and routes the query by the quickest path.

MOBILE WEB SERVICES

The challenge for architects designing software is to be able to seamlessly support mobile devices. Where a software architect could before count on a keyboard, mouse, and monitor together with a hard disk drive, a reasonable amount of memory and processing power, she can no longer count on such a fixed and well-defined target platform. Instead, the architect must now concern herself with whether a display is available at all, and whether the available processing resources are sufficient to provide a reasonable latency to the application. Moreover, as the number and type of mobile devices increase, architects must think about how the nuances between different devices will affect their software.

Web services are an interesting addition to the technology mix for developing mobile applications. The use of Web services allows some, if not most, of the application's business logic to run on remote servers, which are independent of the mobile device's computational resource limitations. This has the added benefit that a variety of mobile devices can effectively access the same functionality or business logic.

CHALLENGES WITH MOBILE

Many challenges exist in the development of mobile applications and systems that are usually not an issue in the development of non-mobile systems. For instance, non-mobile developers rarely think about the ramifications that their application architecture will have on a system's energy consumption. Non-mobile developers also do not usually think about how their application's network utilization will affect the user's monthly wireless subscription bill.

PROXY-BASED MOBILE SYSTEMS

When developing mobile Web services-based applications, a variety of different architectures are possible. A mobile systems architecture that is commonly used is a proxy-based one in which the mobile application communicates only with a proxy server.

The proxy server in turn communicates with and manages the back-end resources, such as Web services.

The mobile device is capable of running multiple applications. Each of these mobile
applications presents a front-end user interface that presents information to users and
captures user input. The application does only enough processing on the user input so that the proxy server can properly interpret the data. Different proxy servers may exist for different applications, or a single server may handle multiple applications. The role of the proxy server is to implement all required business logic to service the needs of the mobile application. The implementation of the business logic may include Web services or other distributed computing resources.

DIRECT MOBILE WEB SERVICE ACCESS

Mobile applications can directly access Web services without an intermediary proxy server. MobileDirectTestServiceInvoke.java is a simple Java program that uses the InvokeService class and directly calls the ChessMaster Web service without using a proxy server for the invocation.

J2ME WEB SERVICES

The Java 2 Micro Edition (J2ME) Web Service specification is an attempt at standardizing programmatic access to Web services from J2ME client applications. Currently, this specification exists as Java Specification Request (JSR) 172 as part the Java Community Process (JCP). More information about JSR-172 can be found at http://www.jcp.org.

Web services can be accessed by J2ME clients today, but involve low-level network APIs that result in non-standard and proprietary implementations. Other specifications and initiatives, such as Java API for XML Processing (JAXP) and Java API for XML-based RPC (JAX-RPC), have addressed high-level and standardized means of accessing XML- based Web services from Java platforms. However, these initiatives do not sufficiently

address the unique requirements and limitations of mobile environments, including footprint, computational resources, and wireless networks.

Prepare the J2ME Web service client and Java Web service
Run KToolbar of J2ME Wireless Toolkit.
Create the new project (Hello).
Specify the Project Name (Hello).
Specify the MIDlet Class Name (HelloMidlet).
Select the Additional APIs - Web Service Access for J2ME (JSR 172).
Create the Web service directory (C:\WTK22\apps\Hello\server).
Create the source directory of Web service (C:\WTK22\apps\Hello\server\src\hello).
Write the property file (build.properties) for Apache Ant (a Java-based build tool) (in C:\WTK22\apps\Hello\server) and these properties must be set according to your environment (project-root=C:/WTK22/apps/Hello/server).
Write the compilation and deployment file (build.xml) (in C:\WTK22\apps\Hello\server)
for Apache Ant and specify the project name according to your environment (project name
= "helloservice").

Create and deploy a Java Web service.

Write the Web service interface class (Hello.java) (in C:\WTK22\apps\Hello\server\src\hello).

```
package hello;

import java.rmi.*;

public interface Hello extends Remote {

public String getHello(String name) throws RemoteException;

}
```

Write the Web service implementation class (HelloImpl.java) (in C:\WTK22\apps\Hello\server\src\hello).

```
package hello;
import java.rmi.RemoteException;
public class HelloImpl implements Hello {
public String getHello(String name) throws
RemoteException {
return "Hello, " + name + "!";
}
}
```

Write the associated descriptor files (all XML files) (in C:\WTK22\apps\Hello\server\src).

o config.xml
o jaxrpc-ri.xml
o web.xml

Compile the service (Hello.java and HelloImpl.java) and generate Web services stubs/ties (C:\WTK22\apps\Hello\server\generated) and WSDL file (C:\WTK22\apps\Hello\server\WEB-INF\classes\helloservice.wsdl) using tools provided
with the Java Web Services Developer Pack:
ant compile
Deploy the web service to the Tomcat server.
o Build the web service to generate a web application archive (C:\WTK22\apps\Hello\server\helloservice.war):
o ant build
o Remove the previously installed Web service (C:\tomcat50-jwsdp\webapps\helloservice.war).
If it is not removed before deployment, the incorrect WSDL (helloservice.wsdl)
might be installed.
o Deploy the web service to the Tomcat server (C:\tomcat50-jwsdp\webapps\helloservice.war):

ant deploy

Verify if the Web service is correctly deployed.

Access http://localhost:8080/helloservice/helloservice.

Click http://localhost:8080/helloservice/helloservice?WSDL

Create the J2ME Web service client

Use the generated WSDL file and the tools built into the Wireless Toolkit to generate the stubs and supporting code used by the MIDlet to access the Web service.

Specify the location of the Web service. (in C:\WTK22\apps\Hello\server\WEB-INF\classes\helloservice.wsdl)

<soap:address location="http://localhost:8080/helloservice/helloservice"/>

Click the Project menu and select Stub Generator.

Specify WSDL Filename or URL (C:\WTK22\apps\Hello\server\WEB-INF\classes\helloservice.wsdl).

Specify Output Package (helloservice) and generate client stubs (in C:\WTK22\apps\Hello\src\helloservice).

Code the MIDlet (HelloMidlet.java) (in C:\WTK22\apps\Hello\src) and associated classes using JAX-RPC to invoke the Web service and JAXP to process the SOAP message.

```
import javax.microedition.midlet.*;
import javax.microedition.lcdui.*;
import java.io.*;
import java.util.*;
import java.rmi.RemoteException;
import helloservice.*;
public class HelloMidlet extends MIDlet implements Runnable, CommandListener {
Hello_Stub service;
Display display;
private Form f;
private StringItem si;
private TextField tf;
private Command sendCommand = new Command("Send", Command.ITEM, 1);
private Command exitCommand = new Command("Exit", Command.EXIT, 1);
String name = "";
```

```
public void startApp() {
display = Display.getDisplay(this);
f = new Form("Hello Client");
tf = new TextField("Send:", "", 30, TextField.ANY);
si = new StringItem("Status:" , " ");
f.append(tf);
f.append(si);
f.addCommand(sendCommand);
f.addCommand(exitCommand);
f.setCommandListener(this);
display.setCurrent(f);
}
```

```
public void pauseApp() {}
public void destroyApp(boolean
unconditional) {}
public void
commandAction(Command c,
Displayable d) {
if (c == sendCommand) {
name = tf.getString();
si.setText("Message sent: " + name);
/*
* Start a new thread so that the remote
invocation won't block
* the process.
*/
new Thread(this).start();
}
if (c == exitCommand) {
notifyDestroyed();
destroyApp(true);
}
}
public void run() {
try {
service = new Hello_Stub();
service._setProperty(Hello_Stub.SESSI
ON_MAINTAIN_PROPERTY, new
Boolean(true));
String msg = service.getHello(name);
si.setText("Message Receive: " + msg);
} catch (Exception exception) {}
}
}
```

REAL WORLD WEB SERVICES DEVELOPMENT AND DEPLOYMENT

Introduction

This document describes how to install Apache Axis. It assumes you already know how to write and run Java code and are not afraid of XML. You should also have an application server or servlet engine and be familiar with operating and deploying to it. If you need an application server, we recommend Jakarta Tomcat. [If you are installing

Tomcat, get the latest 4.1.x version, and the full distribution, not the LE version for Java 1.4, as that omits the Xerces XML parser]. Other servlet engines are supported, provided they implement version 2.2 or greater of the servlet API. Note also that Axis client and server requires Java 1.3 or later.

Enterprise Procurement

Most companies have to buy goods or services from other companies in the course of doing

their business. The procurement can be for physical components such as silicon chips,

plastic components, or power modules, as well as for services such as contract

manufacturing or overnight courier services.

System Functionality and Architecture

The basic functionality of our Enterprise Procurement System (EPS) application is to:

Provide a Web-based interface that allows users to browse a catalog of goods and services from which they can select a particular part number.

Provide a Web-based interface that allows users to enter a part number to be located, the desired quantity, and whether the user is interested in optimizing for cost (if a high-volume product is being built) or for lead time (if a prototype needs to be built rapidly).

Connect to the Web services of a set of vendors and use the entered part number, quantity, and optimization criteria (cost or lead time) to get a price or lead time quote.

Analyze the returned values from the vendor Web services, and select the best choice.

Present the best price or lead-time from the available vendors that meet the user's procurement needs.

If the part cannot be located, consult another Web service to find alternate part numbers that may have similar functionality to that of the first part. Present the alternate part number to the user; otherwise, inform the user that there is no such alternate part.

Running the EPS Application

In this section, we briefly look at the flow through the EPS application from the user's perspective. In the next section, we delve into the actual development of the EPS.

The static HTML page, generated by the EPS.html file, is the main form in which the user enters information about the component to be procured. In the screenshot, information for part number 15151, desired quantity of 4, and optimization criteria of lead-time are specified.

For more details on using Axis, please see the user guide.

Things you need to know before writing a Web Service:

1. Core Java datatypes, classes and programming concepts.
2. What threads are, race conditions, thread safety and sychronization.
3. What a classloader is, what hierarchical classloaders are, and the common causes of a "ClassNotFoundException".
4. How to diagnose trouble from exception traces, what a NullPointerException (NPE) and other common exceptions are, and how to fix them.
5. What a web application is; what a servlet is, where classes, libraries and data go in a web application.
6. How to start your application server and deploy a web application on it.
7. What a network is, the core concepts of the IP protocol suite and the sockets API. Specifically, what is TCP/IP.
8. What HTTP is. The core protocol and error codes, HTTP headers and perhaps the details of basic authentication.
9. What XML is. Not necessarily how to parse it or anything, just what constitutes well-formed and valid XML.

Axis and SOAP depends on all these details. If you don't know them, Axis (or anyone else's Web Service middleware) is a dangerous place to learn. Sooner or later you will be forced to discover these details, and there are easier places to learn than Axis.
If you are completely new to Java, we recommend you start off with things like the Java
Tutorials on Sun's web site, and perhaps a classic book like Thinking in Java, until you have enough of a foundation to be able to work with Axis. It is also useful to have written a simple web application, as this will give you some knowledge of how HTTP works, and how Java application servers integrate with HTTP. Be aware that there is a lot more needed to be learned in order to use Axis and SOAP effectively than the listing
above. The other big area is "how to write internet scale distributed applications". Nobody knows how to do that properly yet, so that you have to learn this by doing.

Step 0: Concepts

Apache Axis is an Open Source SOAP server and client. SOAP is a mechanism for inter-application communication between systems written in arbitrary languages, across the

Internet. SOAP usually exchanges messages over HTTP: the client POSTs a SOAP request, and receives either an HTTP success code and a SOAP response or an HTTP error code. Open Source means that you get the source, but that there is no formal support organisation to help you when things go wrong.
SOAP messages are XML messages. These messages exchange structured information between SOAP systems. Messages consist of one or more SOAP elements inside an envelope, Headers and the SOAP Body. SOAP has two syntaxes for describing the data in these elements, Section 5, which is a clear descendant of the XML RPC system, and XML Schema, which is the newer (and usually better) system. Axis handles the magic of converting Java objects to SOAP data when it sends it over the wire or receives results. SOAP Faults are sent by the server when something goes wrong; Axis converts these to Java exceptions.
SOAP is intended to link disparate systems. It is not a mechanism to tightly bind Java programs written by the same team together. It can bind Java programs together, but not as tightly as RMI or Corba. If you try sending many Java objects that RMI would happily serialize, you will be disappointed at how badly Axis fails. This is by design: if Axis copied RMI and serialized Java objects to byte streams, you would be stuck to a particular version of Java everywhere.

Axis implements the JAX-RPC API, one of the standard ways to program Java services. If you look at the specification and tutorials on Sun's web site, you will understand the API. If you code to the API, your programs will work with other implementations of the API, such as those by Sun and BEA. Axis also provides an extension feature that in many ways extends the JAX-RPC API. You can use these to write better programs, but these will only work with the Axis implementation. But since Axis is free and you get the source, that should not matter.

Axis is compiled in the JAR file axis.jar; it implements the JAX-RPC API declared in the JAR files jaxrpc.jar and saaj.jar. It needs various helper libraries, for logging, WSDL

processing and introspection. All these files can be packaged into a web application, axis.war, that can be dropped into a servlet container. Axis ships with some sample SOAP services. You can add your own by adding new compiled classes to the Axis webapp and registering them.

Before you can do that, you have to install it and get it working.

Step 1: Preparing the webapp

Here we assume that you have a web server up and running on the localhost at port 8080.

If your server is on a different port, replace references to 8080 to your own port number.

In your Application Server installation, you should find a directory into which web applications ("webapps") are to be placed. Into this directory copy the webapps/axis directory from the xml-axis distribution. You can actually name this directory anything you want, just be aware that the name you choose will form the basis for the URL by which clients will access your service. The rest of this document assumes that the default webapp name, "axis" has been used; rename these references if appropriate.

Step 2: Setting up the libraries

In the Axis directory, you will find a WEB-INF sub-directory. This directory contains some basic configuration information, but can also be used to contain the dependencies
and web services you wish to deploy.

Axis needs to be able to find an XML parser. If your application server or Java runtime does not make one visible to web applications, you need to download and add it. Java 1.4 includes the Crimson parser, so you can omit this stage, though the Axis team prefer
Xerces.

To add an XML parser, acquire the JAXP 1.1 XML compliant parser of your choice. We recommend Xerces jars from the xml-xerces distribution, though others mostly work. Unless your JRE or app server has its own specific requirements, you can add the parser's libraries to axis/WEB-INF/lib. The examples in this guide use Xerces. This guide adds xml-apis.jar and xercesImpl.jar to the AXISCLASSPATH so that Axis can find the parser (see below).

If you get ClassNotFound errors relating to Xerces or DOM then you do not have an XML parser installed, or your CLASSPATH (or AXISCLASSPATH) variables are not correctly configured.

Tomcat 4.x and Java 1.4

Java 1.4 changed the rules as to how packages beginning in java.* and javax.* get loaded. Specifically, they only get loaded from endorsed directories. jaxrpc.jar and saaj.jar contain javax packages, so they may not get picked up. If happyaxis.jsp (see below) cannot find the relevant packages, copy them from axis/WEB-INF/lib to CATALINA_HOME/common/lib and restart Tomcat.

WebLogic 8.1

WebLogic 8.1 ships with webservices.jar that conflicts with Axis' saaj.jar and prevents Axis 1.2 from working right out of the box. This conflict exists because WebLogic uses an older definition of javax.xml.soap.* package from Java Web Services Developer Pack Version 1.0, whereas Axis uses a newer revision from J2EE 1.4.

However, there are two alternative configuration changes that enable Axis based web services to run on Weblogic 8.1.

In a webapp containing Axis, set <prefer-web-inf-classes> element in WEB-INF/weblogic.xml to true. An example of weblogic.xml is shown below:

<weblogic-web-app>
<container-descriptor>
<prefer-web-inf-classes>true</prefer-web-inf-classes>
</container-descriptor>
</weblogic-web-app>

o If set to true, the <prefer-web-inf-classes> element will force WebLogic's classloader to load classes located in the WEB-INF directory of a web application in preference to application or system classes. This is a recommended approach since it only impacts a single web module.

In a script used to start WebLogic server, modify CLASSPATH property by placing Axis's saaj.jar library in front of WebLogic's webservices.jar.
o NOTE: This approach impacts all applications deployed on a particular WebLogic instance and may prevent them from using WebLogic's web services.
For more information on how Web Logic's class loader works, see Web Logic Server Application Classloading.
Step 3: starting the web server
This varies on a product-by-product basis. In many cases it is as simple as double clicking on a startup icon or running a command from the command line.

Step 4: Validate the Installation

After installing the web application and dependencies, you should make sure that the server is running the web application.

Look for the start page

Navigate to the start page of the webapp, usually http://127.0.0.1:8080/axis/, though of
course the port may differ.

You should now see an Apache-Axis start page. If you do not, then the webapp is not actually installed, or the appserver is not running.

Validate Axis with happyaxis

Follow the link Validate the local installation's configuration

This will bring you to happyaxis.jsp a test page that verifies that needed and optional libraries are present. The URL for this will be something like
http://localhost:8080/axis/happyaxis.jsp

If any of the needed libraries are missing, Axis will not work.

You must not proceed until all needed libraries can be found, and this validation

page is happy.

Optional components are optional; install them as your need arises. If you see nothing but

an internal server error and an exception trace, then you probably have multiple XML

parsers on the CLASSPATH (or AXISCLASSPATH), and this is causing version

confusion. Eliminate the extra parsers, restart the app server and try again.

Look for some services

From the start page, select View the list of deployed Web services. This will list all registered Web Services, unless the servlet is configured not to do so. On this page, you should be able to click on (wsdl) for each deployed Web service to make sure that your web service is up and running.

Note that the 'instant' JWS Web Services that Axis supports are not listed in this listing here. The install guide covers this topic in detail.

Test a SOAP Endpoint

Now it's time to test a service. Although SOAP 1.1 uses HTTP POST to submit an XML request to the endpoint, Axis also supports a crude HTTP GET access mechanism, which is useful for testing. First let's retrieve the version of Axis from the version endpoint, calling the getVersion method:

http://localhost:8080/axis/services/Version?method=getVersion
This should return something like:

```
<?xml version="1.0" encoding="UTF-8" ?>
<soapenv:Envelope
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
<soapenv:Body>
<getVersionResponse
soapenv:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
<getVersionReturn
xsi:type="xsd:string">
Apache Axis version: 1.1 Built on Apr 04, 2003 (01:30:37 PST)
</getVersionReturn>
</getVersionResponse>
</soapenv:Body>
</soapenv:Envelope>
```

The Axis version and build date may of course be different.

Test a JWS Endpoint

Now let's test a JWS web service. Axis' JWS Web Services are java files you save into

the Axis webapp anywhere but the WEB-INF tree, giving them the .jws extension.

When someone requests the .jws file by giving its URL, it is compiled and executed.

The user guide covers JWS pages in detail.

To test the JWS service, we make a request against a built-in example, EchoHeaders.jws

(look for this in the axis/ directory).

Point your browser at http://localhost:8080/axis/EchoHeaders.jws?method=list.

This should return an XML listing of your application headers, such as

```
<?xml version="1.0" encoding="UTF-8" ?>
<soapenv:Envelope
xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
<soapenv:Body>
<listResponse
soapenv:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
<listReturn xsi:type="soapenc:Array"
soapenc:arrayType="xsd:string[6]"
xmlns:soapenc="http://schemas.xmlsoap.org/soap/encoding/">
<item>accept:image/gif, image/x-xbitmap, image/jpeg, image/pjpeg, */*</item>
<item>accept-language:en-us</item>
<item>accept-encoding:gzip, deflate</item>
<item>user-agent:Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1)</item>
<item>host:localhost:8080</item>
<item>connection:Keep-Alive</item>
</listReturn>
</listResponse>
</soapenv:Body>
</soapenv:Envelope>
```

Again, the exact return values will be different, and you may need to change URLs to

correct any host, port and webapp specifics.

So far you have got Axis installed and working--now it is time to add your own Web

Service.

The process here boils down to (1) get the classes and libraries of your new service into the Axis WAR directory tree, and (2) tell the AxisEngine about the new file. The latter is done by submitting an XML deployment descriptor to the service via the Admin web service, which is usually done with the AdminClient program or the <axis-admin> Ant task. Both of these do the same thing: they run the Axis SOAP client to talk to the Axis administration service, which is a SOAP service in its own right. It's also a special SOAP service in one regard--it is restricted to local callers only (not remote access) and is password protected to stop random people from administrating your service. There is a default password that the client knows; if you change it then you need to pass the new password to the client.

The first step is to add your code to the server.

In the WEB-INF directory, look for (or create) a "classes" directory (i.e. axis/WEB-INF/classes). In this directory, copy the compiled Java classes you wish to install, being careful to preserve the directory structure of the Java packages.

If your classes services are already packaged into JAR files, feel free to drop them into the WEB-INF/lib directory instead. Also add any third party libraries you depend on into the same directory.

After adding new classes or libraries to the Axis webapp, you must restart the webapp. This can be done by restarting your application server, or by using a server-specific mechanism to restart a specific webapp.

Note: If your web service uses the simple authorization handlers provided with xml-axis (this is actually not recommended as these are merely illustrations of how to write a handler than intended for production use), then you will need to copy the corresponding users.lst file into the WEB-INF directory.

Step 6: Deploying your Web Service

The various classes and JARs you have just set up implement your new Web Service. What remains to be done is to tell Axis how to expose this web service. Axis takes a Web Service Deployment Descriptor (WSDD) file that describes in XML what the service is, what methods it exports and other aspects of the SOAP endpoint.

The users guide and reference guide cover these WSDD files; here we are going to use one from the Axis samples: the stock quote service.

Classpath setup

In order for these examples to work, java must be able to find axis.jar, commons-discovery.jar, commons-logging.jar, jaxrpc.jar, saaj.jar, log4j-1.2.8.jar (or whatever is appropriate for your chosen logging implementation), and the XML parser jar file or files (e.g., xerces.jar). These examples do this by adding these files to AXISCLASSPATH and then specifying the AXISCLASSPATH when you run them. Also for these examples, we have copied the xml-apis.jar and xercesImpl.jar files into the AXIS_LIB directory. An alternative would be to add your XML parser's jar file directly to the AXISCLASSPATH variable or to add all these files to your CLASSPATH variable.

On Windows, this can be done via the following. For this document we assume that you have installed Axis in C:\axis. To store this information permanently in WinNT/2000/XP you will need to right click on "My Computer" and select "Properties". Click the "Advanced" tab and create the new environmental variables. It is often better to use WordPad to create the variable string and then paste it into the appropriate text field.

```
set AXIS_HOME=c:\axis
set AXIS_LIB=%AXIS_HOME%\lib
set AXISCLASSPATH=%AXIS_LIB%\axis.jar;%AXIS_LIB%\commons-discovery.jar;
%AXIS_LIB%\commons-logging.jar;%AXIS_LIB%\jaxrpc.jar;%AXIS_LIB%\saaj.jar;

%AXIS_LIB%\log4j-1.2.8.jar;%AXIS_LIB%\xml-
apis.jar;%AXIS_LIB%\xercesImpl.jar
```

Apcahe Axis2

Apache Axis2 is a robust, Java-based web services engine designed to process and route SOAP messages, JSON, and RESTful APIs. Organizations implement Axis2 to unify their multi-protocol communication, allowing a single Java codebase to serve classic SOAP, JSON-RPC, REST, and AI-driven framework traffic simultaneously

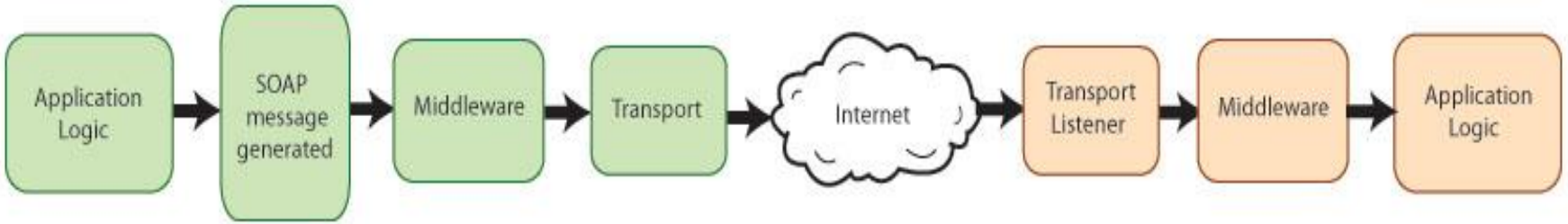
https://axis.apache.org/axis2/java/core/docs/userguide.html

The Apache Axis2 project is a Java-based implementation of both the client and server sides of the Web services equation. Designed to take advantage of the lessons learned from Apache Axis 1.0, Apache Axis2 provides a complete object model and a modular architecture that makes it easy to add functionality and support for new Web services-related specifications and recommendations.

Axis2 enables you to easily perform the following tasks:

- Send SOAP messages
- Receive and process SOAP messages
- Receive and process JSON messages
- Create a Web service out of a plain Java class
- Create implementation classes for both the server and client using WSDL
- Easily retrieve the WSDL for a service
- Send and receive SOAP messages with attachments
- Create or utilize a REST-based Web service

To understand Axis2 and what it does, you must have a good idea of the life cycle of a Web services message. Typically, it looks something like this:



The sending application creates the original SOAP message, an XML message that consists of headers and a body. If the system requires the use of WS* recommendations such as WS-Addressing or WS-Security, the message may undergo additional processing before it leaves the sender. Once the message is ready, it is sent via a particular transport such as HTTP, JMS, and so on.

eclipse-workspace - axis2services/StudentService.wsdl - Eclipse IDE

File Edit Navigate Search Project WSDL Editor Run Design Window Help

Project Explorer

- axis2services
  - Deployment Descriptor: axis2services
  - JAX-WS Web Services
    - Service Endpoint Interfaces
    - Web Services
  - Java Resources
    - src/main/java
      - com.axis2services
        - Student.java
        - StudentServiceImpl.java
    - Libraries
      - JRE System Library [JavaSE-21]
      - Maven Dependencies
      - Server Runtime [Apache Tomca
  - Deployed Resources
  - build
  - src
    - main
  - target
  - pom.xml
  - student.xsd
  - StudentService.wsdl
- DemoR
- demoxml
- FirstWS
- myrest
- RestApp
- RestDemo
- RMIXMLExample
- SAX Parser example
- Servers

StudentResul... | services.xml | StudentServi... | axis2servic... | student.xsd | StudentServi...

```
1  <wsdl:definitions
2      xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/"
3      xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
4      xmlns:xs="http://www.w3.org/2001/XMLSchema"
5      xmlns:tns="http://student.example.com"
6      targetNamespace="http://student.example.com">
7      <wsdl:types>
8          <xs:schema>
9              <xs:import namespace="http://student.example.com" schemaLocation="student.xsd"/>
10         </xs:schema>
11     </wsdl:types>
12     <wsdl:message name="GetStudentRequest">
13         <wsdl:part name="parameters" element="tns:GetStudentRequest"/>
14     </wsdl:message>
15     <wsdl:message name="GetStudentResponse">
16         <wsdl:part name="parameters" element="tns:GetStudentResponse"/>
17     </wsdl:message>
18     <wsdl:portType name="StudentService">
19         <wsdl:operation name="getStudent">
20             <wsdl:input message="tns:GetStudentRequest"/>
21             <wsdl:output message="tns:GetStudentResponse"/>
22         </wsdl:operation>
23     </wsdl:portType>
24     <wsdl:binding name="StudentServiceSOAP" type="tns:StudentService">
25         <soap:binding style="document"
26             transport="http://schemas.xmlsoap.org/soap/http"/>
27         <wsdl:operation name="getStudent">
28             <soap:operation soapAction="getStudent"/>
```

Design | Source

Outline

- Imports
- Types
- Services
- Bindings
- Port Type
- Messages

Problems | Servers | Terminal | Data Source Explorer | Properties | Console

Tomcat v9.0 Server at localhost [Started, Restart]

axis2services(axis2services-0.0.1-SNAPSHOT) [Synchronized]

axis2services

Apache Tomcat/9.0.115

If you're seeing this, you've successfully installed Tomcat. Congratulations!

Recommended Reading:

Security Considerations How-To
Manager Application How-To
Clustering/Session Replication How-To

Server Status
Manager App
Host Manager

Developer Quick Start

Tomcat Setup
First Web Application
Realms & AAA
JDBC DataSources
Examples
Servlet Specifications
Tomcat Versions

Managing Tomcat

For security, access to the manager webapp is restricted. Users are defined in:

```
$CATALINA_HOME/conf/tomcat-users.xml
```

In Tomcat 9.0 access to the manager application is split between different users.
Read more...

Release Notes
Changelog

Documentation

Tomcat 9.0 Documentation
Tomcat 9.0 Configuration
Tomcat Wiki

Find additional important configuration information in:

```
$CATALINA_HOME/RUNNING.txt
```

Developers may be interested in:

Tomcat 9.0 Bug Database

Getting Help

FAQ and Mailing Lists

The following mailing lists are available:

tomcat-announce
Important announcements, releases, security vulnerability notifications. (Low volume).

tomcat-users
User support and discussion

taglibs-user
User support and discussion for Apache Taglibs

tomcat-dev

Welcome!

Welcome to the new generation of Axis. If you can see this page you have successfully deployed the Axis2 Web Application. However, to ensure that Axis2 is properly working, we encourage you to click on the validate link.

- Services
  View the list of all the available services deployed in this server.
- Validate
  Check the system to see whether all the required libraries are in place and view the system information.
- Administration
  Console for administering this Axis2 installation.

The Apache Software Foundation
http://www.apache.org/

AXIS2

Back Home | Refresh

Axis2 Happiness Page

Examining webapp configuration

Essential Components

Found Apache-Axis (org.apache.axis2.transport.http.AxisServlet)
  at D:\apache-tomcat-9.0.115\webapps\axis2\WEB-INF\lib\axis2-transport-http-1.8.2.jar
Found Jakarta-Commons Logging (org.apache.commons.logging.Log)
  at D:\apache-tomcat-9.0.115\webapps\axis2\WEB-INF\lib\commons-logging-1.2.jar
Found Streaming API for XML (javax.xml.stream.XMLStreamReader)
  at an unknown location
Found Streaming API for XML implementation (org.codehaus.stax2.XMLStreamWriter2)
  at D:\apache-tomcat-9.0.115\webapps\axis2\WEB-INF\lib\stax2-api-4.2.1.jar

The core axis2 libraries are present.

Note: Even if everything this page probes for is present, there is no guarantee your Axis Service will work, because there are many configuration options that we do not check for. These tests are *necessary* but not *sufficient*

Examining Version Service

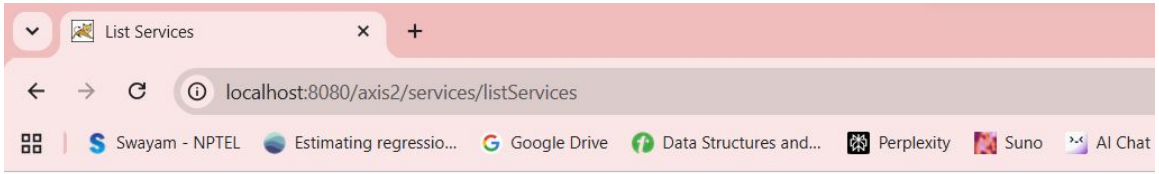
Found Axis2 default Version service and Axis2 is working properly.

Now you can drop a service archive in axis2/WEB-INF/services. Following output was produced while invoking Axis2 version service

Examining Application Server

Servlet version 4.0
Platform Apache Tomcat/9.0.115

Examining System Properties



The Apache Software Foundation
http://www.apache.org/

AXIS2

Back Home | Refresh

Available services

StudentService

Service Description : Student Details SOAP Service

Service EPR : http://localhost:8080/axis2/services/StudentService

Service Status : Active

Available Operations

- getAllStudents

Version

Service Description : This service is to get the running Axis version

Service EPR : http://localhost:8080/axis2/services/Version

Service Status : Active

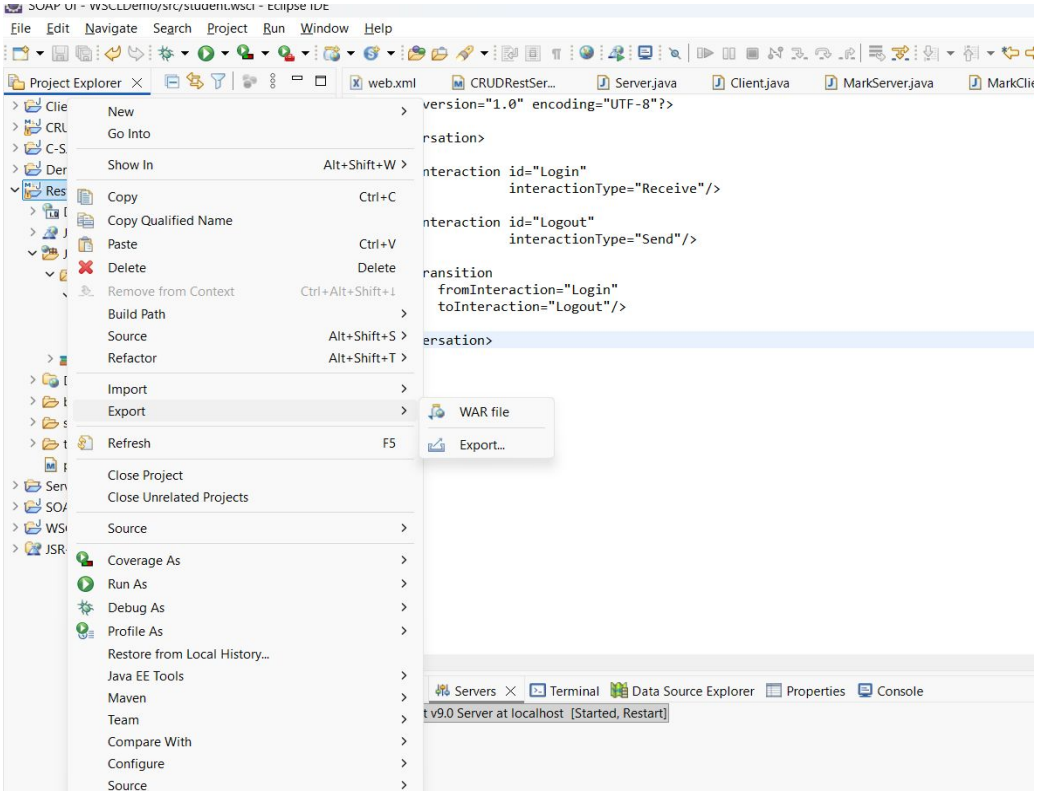
Available Operations

- getVersion

Feature	Apache Axis2	Apache Tomcat 9
Primary Category	Web Services Engine / Framework	Web Server & Servlet Container
Core Function	Generates, parses, and processes SOAP and REST messages.	Listens for HTTP requests and manages the lifecycle of Servlets and JSPs.
Operation Level	Runs at the application layer inside a servlet container.	Runs as a standalone server environment.
Primary Formats	Handles WSDL, SOAP, XML, and JSON payloads.	Handles HTTP requests, HTML, CSS, and basic web traffic.
Packaging Extension	Packages web services as `.aar` (Axis Archive) or `.war` files.	Hosts standard `.war` (Web Archive) files and exploded directories.

Deploy the Created and tested Web services

1. Right click on project > Export > War file
2. Paste the War file into > apache-tomcat-9.0\webapps\
3. Run apache-tomcat-9.0\bin\startup.bat
4. Tomcat automatically extracts: StudentResultService.war (Your War file)
5. Open: http://localhost:8080/StudentResultService/StudentResult?wsdl
6. If WSDL appears, deployment is successful.



Deploy on Another Computer

1. Copy StudentResultService.war to another machine having:

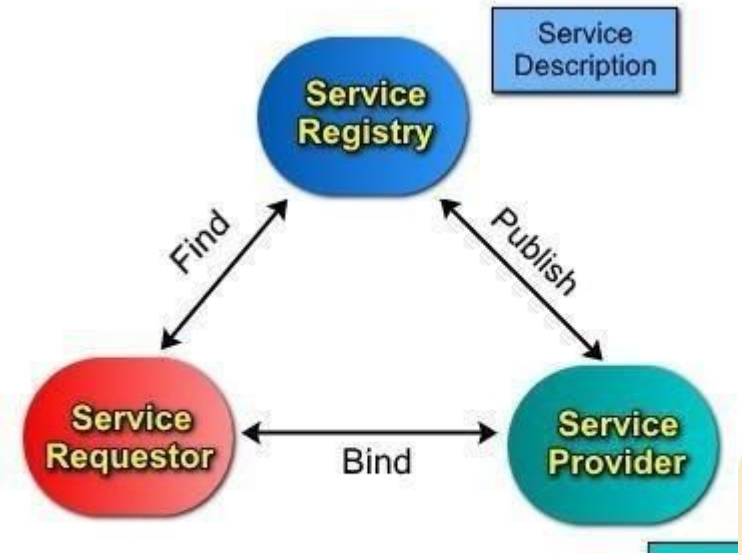
- JDK 8
- Tomcat 9

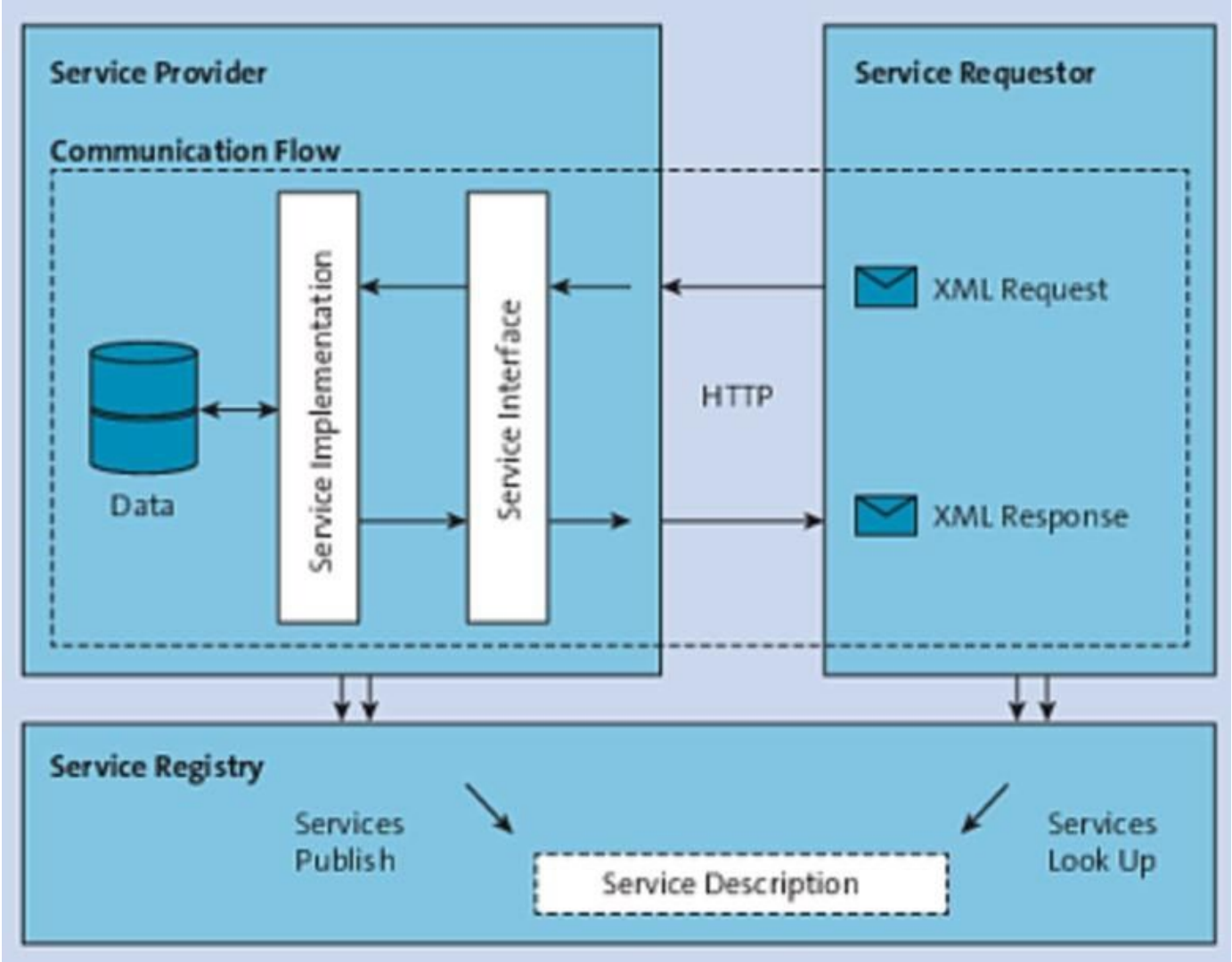
2. Place WAR in: webapps
3. Start Tomcat.
4. Service becomes available as:
   http://IP_Address:8080/StudentResultService/StudentResult?wsdl
   Example:
   http://192.168.1.10:8080/StudentResultService/StudentResult?wsdl
5. **Verify Using SoapUI After Deployment**
6. Create a new SOAP Project.
7. Enter: http://localhost:8080/StudentResultService/StudentResult?wsdl
8. SoapUI should generate the request automatically and return the response.

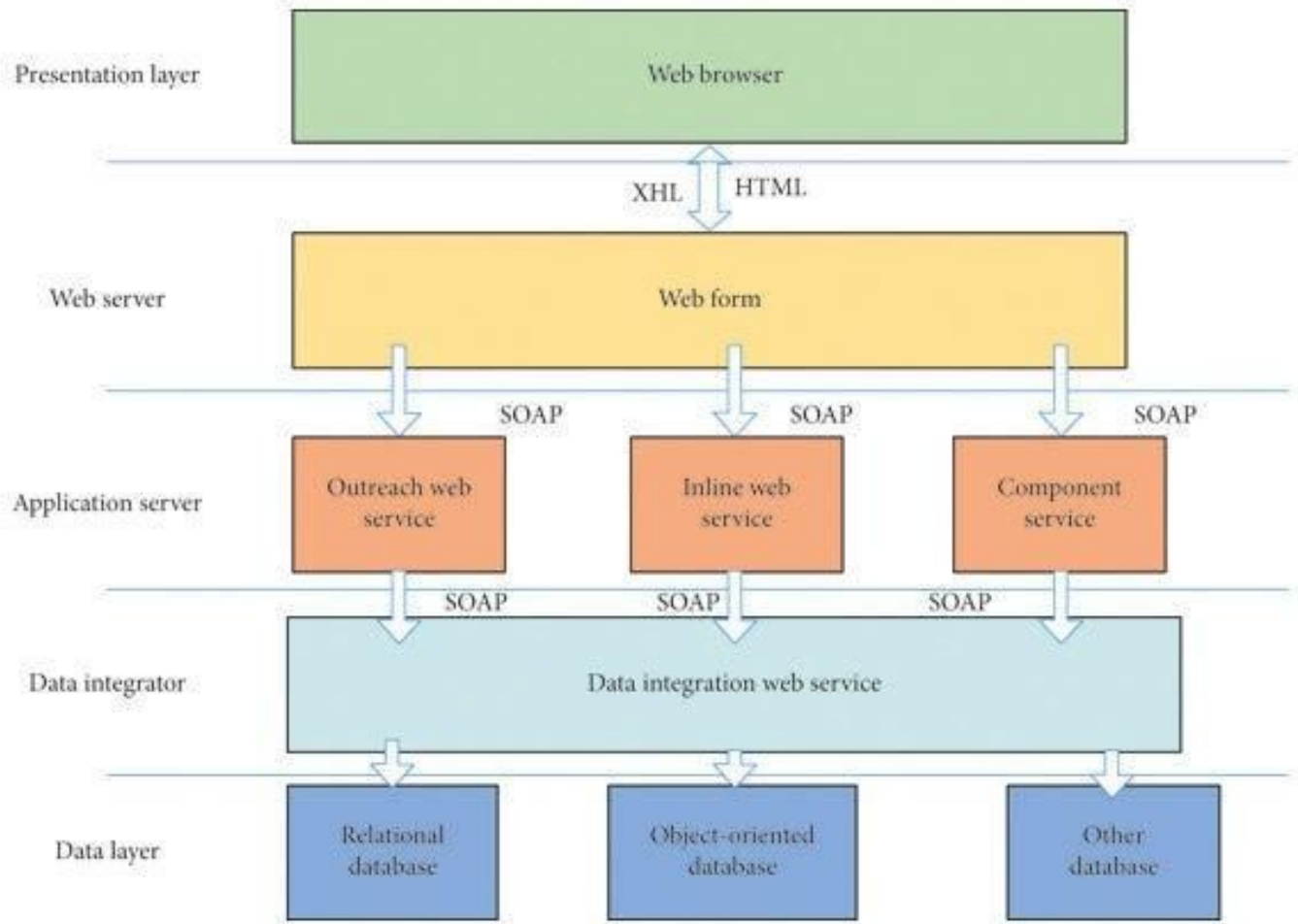
Building Real World Enterprise Web Service and Applications: Real World Web Service Application Development

Enterprise Web Service Application Development refers to the process of designing, developing, deploying, and maintaining web services that enable communication and data exchange between different business applications over a network.

Architecture of an Enterprise Web Service




Service Provider
Service Requestor
Communication Flow
Service Implementation
Service Interface
XML Request
HTTP
Data
XML Response
Service Registry
Services
Publish
Service Description
Services
Look Up


Presentation layer
Web browser
XHL
HTML
Web server
Web form
SOAP
SOAP
SOAP
Application server
Outreach web service
Inline web service
Component service
SOAP
SOAP
SOAP
Data integrator
Data integration web service
Data layer
Relational database
Object-oriented database
Other database

Components

1. **Service Provider**
   - Creates and publishes the web service.
   - Makes the service available to clients.
2. **Service Consumer (Client)**
   - Uses the web service by sending requests and receiving responses.
3. **Service Registry**
   - Stores information about available services.
   - Example: UDDI Registry.
4. **Communication Protocol**
   - SOAP or REST.
5. **Data Format**
   - XML (SOAP)
   - XML/JSON (REST)

Steps in Real World Web Service Development

1. Requirement Analysis

- Identify business requirements.
- Determine services to be exposed.

Example:

Student Management System

Services:

- Add Student
- Search Student
- Update Student
- Delete Student

2. Service Design

Design operations and messages.

Example:

1. addStudent()
2. getStudent()
3. updateStudent()
4. deleteStudent()

Define:

- Input parameters
- Output parameters
- Error handling

3. WSDL Creation

WSDL describes:

- Operations
- Messages
- Data types
- Endpoints

Example:

<operation name="getStudent">

4. Service Implementation

Using Java:

```
@WebService
public class StudentService {

   @WebMethod
   public String getStudent(int id) {
       return "Student Found";
   }
}
```

5. Deployment

Deploy on server:

- Apache Tomcat
- GlassFish
- JBoss
- WebLogic

Generate:

http://localhost:8080/StudentService?wsdl

6. Testing

Tools:

- SoapUI
- Postman
- Browser

Test:

- Valid requests
- Invalid requests
- Performance

7. Security Implementation

Enterprise applications require security.

Techniques:

- Authentication
- Authorization
- SSL/TLS
- WS-Security

Example:

Username + Password
HTTPS

8. Monitoring and Maintenance

Monitor:

- Service availability
- Response time
- Error logs

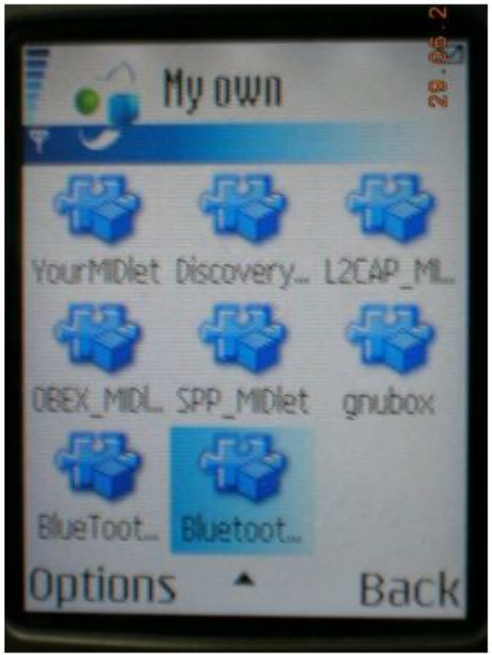
Update services when business requirements change.

Mobile Web Services

- Middleware It consists of several modules (Fig. 3). In general, the middleware communicates in between the web server, where the services are deployed, and the mobile client, to facilitate the invocation of services by mobile clients dynamically. In this section we have discussed how all these modules work together and how they interact with the mobile client running on mobile device
- Middleware: This is deployed on a computer that is Bluetooth-enabled, Wi-Fi enabled and also connected to GSM network. Because of limited coverage of Bluetooth and Wi-Fi connections, when a mobile device is out of the coverage area of the middleware available on the local server, it should connect to the middleware that is available on the Internet through GPRS connection. A web server is integrated into the middleware system, where some services that provide localized information are deployed. The server is connected to the GSM network for authenticating the mobile devices. • Mobile Devices: Different types of mobile devices like PDA, mobile phones, palmtops etc, can consume the services. The minimum requirement is that these mobile devices need to have Bluetooth, Wi-Fi or GPRS connectivity.

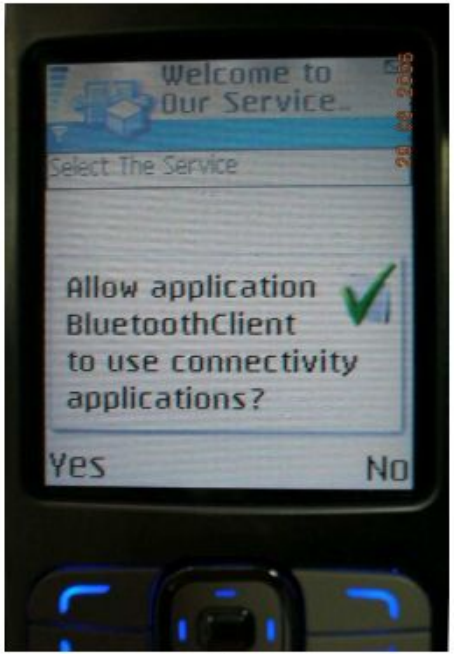
people like to have localized information and location specific services to be displayed on their mobile devices. At the same time, people would like to invoke from their mobile devices the Internet based services that provide solution for complex problem while on the move. Location specific service means that the same consumer with the same portable device will receive different sets of services depending on his current location. Our proposed architecture provides location-based services [1] and services available on the internet to mobile users, irrespective of the type of the device. The mobile user is also free to invoke services of his choice from a wide array of available services. To present a typical scenario, suppose a new visitor has entered into Jadavpur University with a mobile phone like Nokia 6600 or Nokia N series or a PDA that is either Bluetooth enabled or Wi-Fi enabled where a client application is already installed. If the visitor starts that client application, after a few seconds, he would get the following messages on the screen

A little while after the first screen, a prompt message would appear on the screen asking the mobile user's consent to accept connection from the middleware. Once the user presses the 'yes' button, names of a number of services would appear on the next screen.

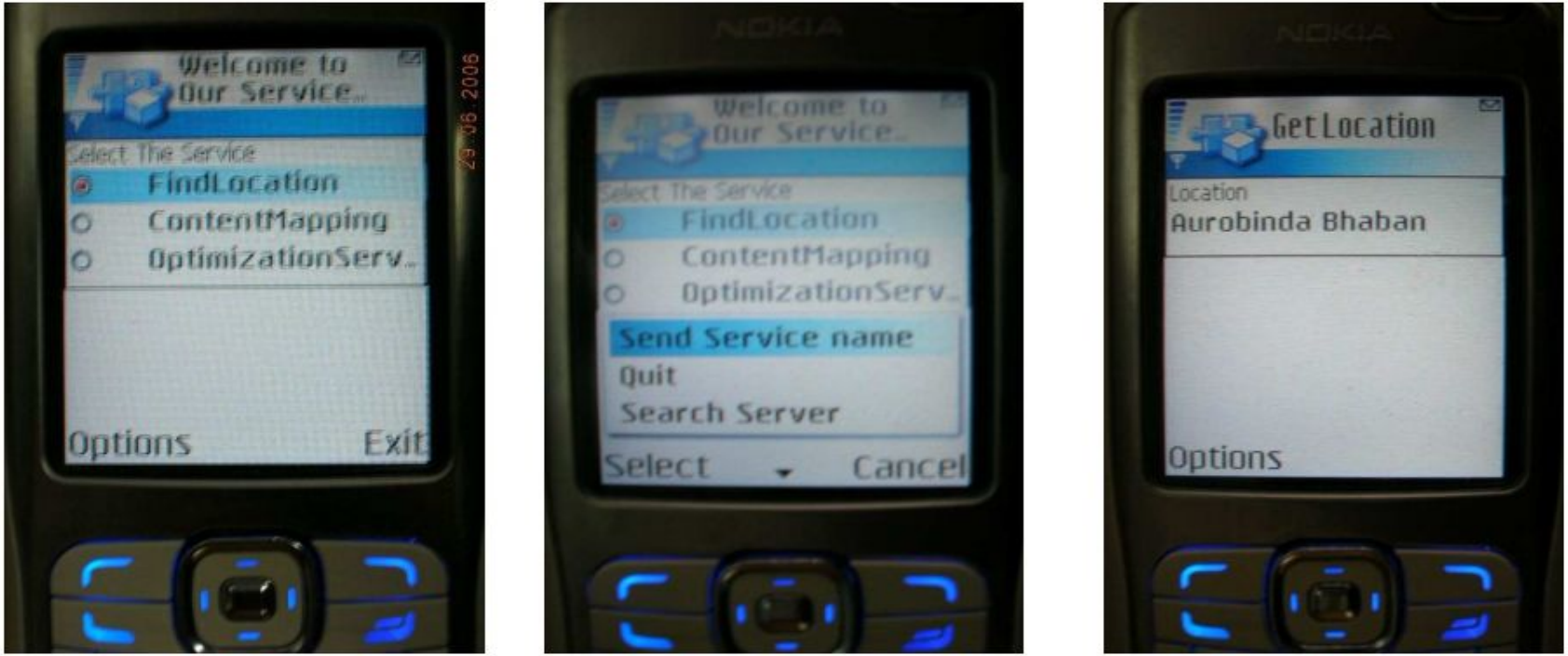


1

2



3



4 5 6

Figure 1: Messages that appear on the mobile screen for making connection, service advertisement and invocation of service

Now the user selects one of them (say, Find Location that gives the client the name of location he is currently standing on) and presses the 'send service name' button, the corresponding service would be executed by the middleware and the result sent back to the mobile device.

Internet based web services and web contents to the mobile devices and the mobile device is required to send the request for a service by sending the URL of the service. In [7], authors suggested a HTML/WSDL converter system that would receive the URL for requested service from the mobile client. The converter will generate the WSDL file so that mobile devices can access that web content as a web service. Only those mobile devices that have SOAP message processing capabilities can utilize this. They have not shown how to invoke the services from mobile devices that do not have the SOAP message processing capabilities or how to discover the services. In [8], authors have proposed proxy based distributed architecture where a local access point advertises the services that are available on the local web server. However, they have not highlighted how heterogeneous clients like Bluetooth-enabled device, Wi-Fi enabled device or a device that possesses GPRS connectivity would be able to make connection with local access point and how they would be able to invoke the services when mobile device is out of the coverage area of Wi-Fi connection of the local access point. Another crucial point, overlooked by all previous researchers, is the need to authenticate the user before providing him the services

a mobile device would be able to invoke the services from anywhere through GPRS connection even if there is no middleware system in the vicinity of the mobile device. Here the system where middleware is deployed is called the Base Station(BS). Middleware has been developed in Java and it consists of several modules. All these modules have been described in detail in the following section. A web server (Tomcat 5.0 [10]) resides on the Base Station to deploy the local web services that provide localized information. These local web services are developed by Java Web Service Developer Pack 1.6 (JWSDP 1.6) tool [11] and deployed on Tomcat server. The client program that runs on the mobile device is developed in J2ME [12]. Some of the Base Stations have global IP [13] to make them available on Internet. The middleware that is deployed on this system is termed as web based middleware. The web based middleware has been developed using WML [14] and JSP [14] and is also deployed on Tomcat web server.

Middleware

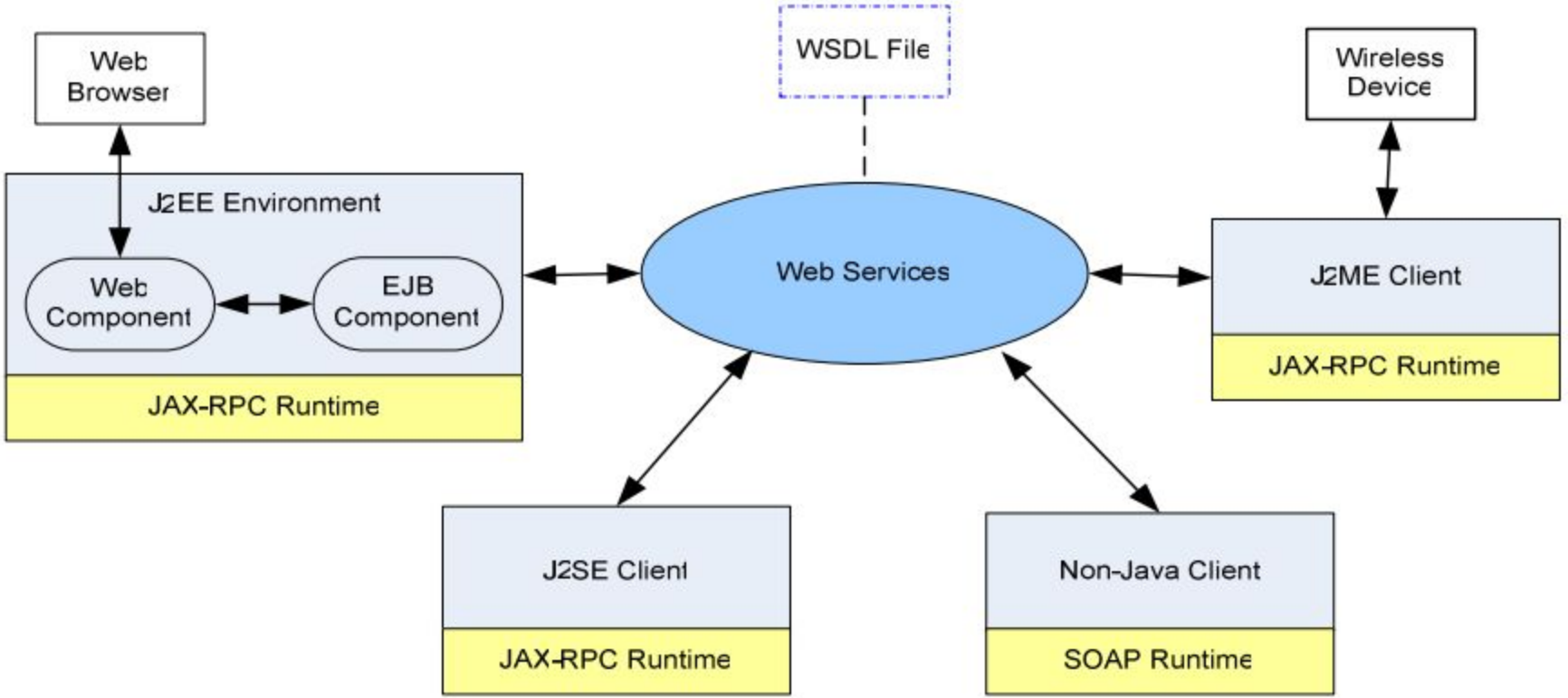
the middleware communicates in between the web server, where the services are deployed, and the mobile client, to facilitate the invocation of services by mobile clients dynamically. In this section we have discussed how all these modules work together and how they interact with the mobile client running on mobile device.

Main Module (MM) It simultaneously invokes Bluetooth Connection Module and Wi-Fi connection module and continues to do this every 30 seconds.

Web Based Middleware It consists of three modules – Main Module, Authentication Module and Service Advertising & Invocation Module. Interaction between these modules and interaction of these modules with the client program is identical to what is shown in figure 3. If the mobile client cannot find any Wi-Fi or Bluetooth enabled Base Station in its vicinity, client application displays a well known URL for the web based middleware and instructs the user to type in the URL on micro web browser to connect to the middleware. For example, the URL can be http://250.68.66.147:8080/middleware/main_module.wml As we have already pointed out this middleware has been developed using WML and JSP. Once the Main Module has received the phone number from the mobile device, it invokes AM for ensuring that phone number is correct and makes an entry into the local database. It then invokes the SAIM to provide the service to the mobile device.

Mobile Client Application It is developed using J2ME (Java 2 Mobile Edition). To successfully run the client application on a mobile device inside the cell of a Base station, the device should have the support of Bluetooth API (JSR 82) and Wireless Messaging API (JSR 120). If a device does not have support for Bluetooth API or Wireless Messaging API, client program displays the URL of a WML web page to be run on the micro web browser. Runtime Interaction between Middleware and Mobile Client: In this section we show the set of events and exchange of messages that takes place between the middleware and the mobile client and exchange of messages between different modules of the middleware during invocation of service by the mobile client.

Various clients running on different types of platforms can all access the Web service. We can design and develop a full functionalities client using J2EE technologies, or a rich Java application by standard J2SE, or even a light-weight application client which running on a J2ME-enabled phone connected to the Internet shows how these different types of clients might access the Web service.


Web Browser
WSDL File
Wireless Device
J2EE Environment
Web Component
EJB Component
JAX-RPC Runtime
Web Services
J2ME Client
JAX-RPC Runtime
J2SE Client
JAX-RPC Runtime
Non-Java Client
SOAP Runtime

Reasons to implement Web Service with J2ME

- First of all, due to the facts of limited amount of memory, GUI capabilities, process power, together with network connection, bandwidth limitation and so forth, design a good J2ME client is becoming a issue that concern not only basic design consideration for the wire device, but also other aspects, like how to limit the data exchange rate, reduce the amount of data transferred, simplify the user GUI design, take the data type exchanged that require less pre- and post-processing and etc. Secondly, the traditional SOAP engine such as Apache Axis is far too large and resource-intensive to work on small device.
- It is not possible to expect the mobile device to work with MB sized package which is originally designed for desktop clients and servers. Therefore, the J2ME client typically uses only a small set of the JAX-RPC API. One of such subset used in the thesis is kSOAP [23], an open source project from Enhydra. It is a lightweight SOAP implementation especially suitable for J2ME program. Unlike the common SOAP packages which contain hundreds of classes, kSOAP is combined into a single jar file which takes up less than 42K and makes it become a lightweight SOAP implementation to enable SOAP and XML application to run within a low-memory virtual machine on a micro device.

Locate and Access the Service

There are three principal modes for a client to locate and access a Web service: static stub, dynamic proxy, and dynamic invocation interface (DII) [22] Static Stub – The static stub classes are generated from the JAX-RPC runtime toolkit, e.g. Apache Axis, wscompile, to enable the client to communicate with the service. Contrast to the remote “skeleton” object in the server, the stub is a local object that acts as a proxy for the service endpoint. Because this stub is created before runtime (by the IDE), it is called a static stub. The stub takes the main responsibilities to convert a request from the client to a SOAP message and send it to the service. It also converts the response from the service endpoint to a proper format understandable by the client. Figure 4-2-2 shows how the client use static stub produced by JAX-PRC toolkit to communicate with the server

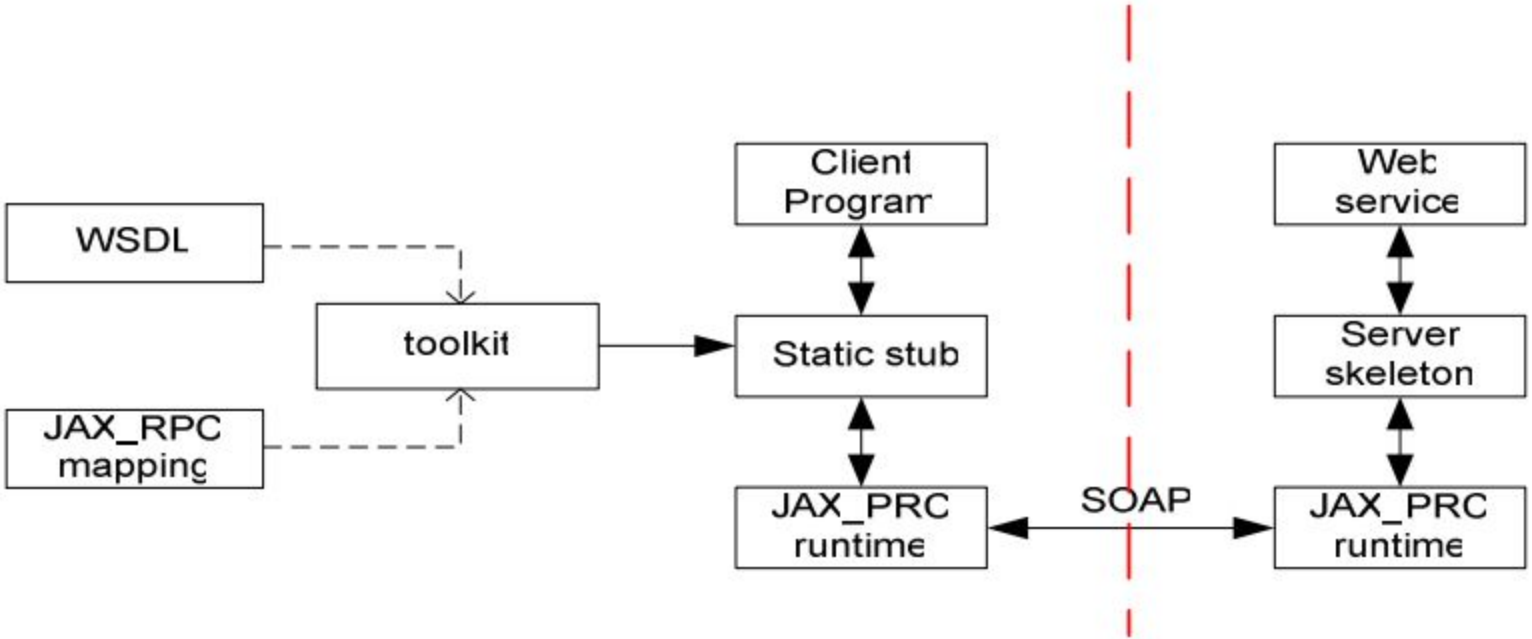


Figure 4-2- 2 Static stub mode

Dynamic Proxy –As the same as the static stub, dynamic proxy provides a method for the client to access the service but in a more dynamic fashion. Dynamic proxy client supports a service endpoint interface dynamically at runtime without requiring any code generation of a stub class during development. The client gets information about the service by looking up its WSDL document and invokes the service directly

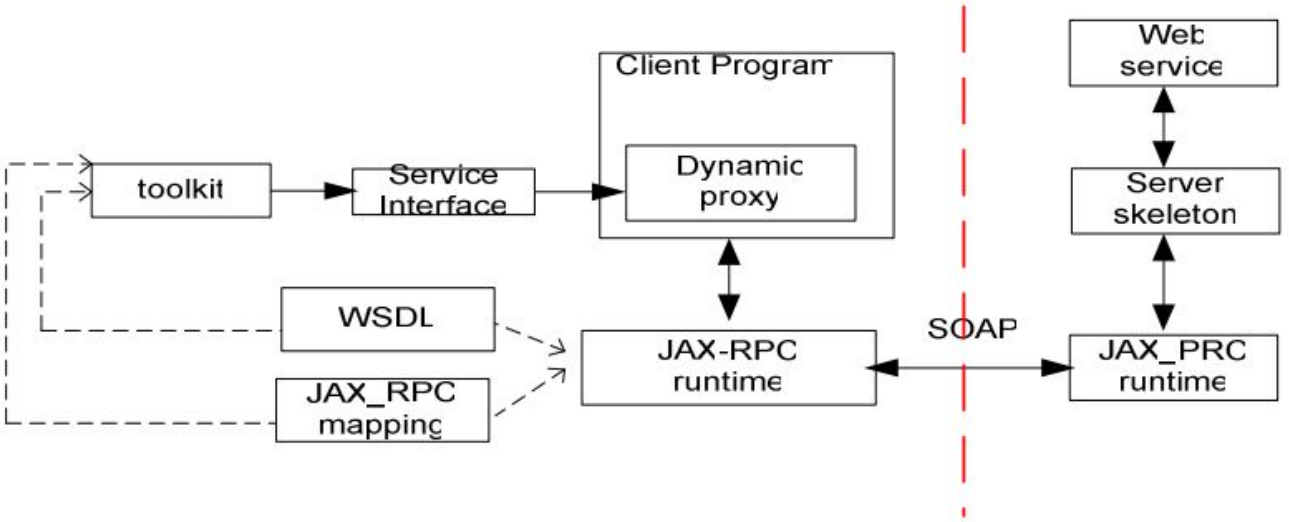


Figure 4-2- 3 Dynamic proxy mode

Dynamic Invocation Interface (DII) – a good reason to use dynamic invocation is to support asynchronous communications, which allow the client application sends the request and free to do other work. It is used at compile time when a client does not have the knowledge about the remote service at the time the client application was written. Compare to the static proxy and dynamic proxy, DII doesn't need prior definition of a service endpoint interface, even the signature of the remote procedure or the name of the service is unknown. All the client knows is about a WSDL file. Obviously the DII client will be much complicated than the other two types of clients.

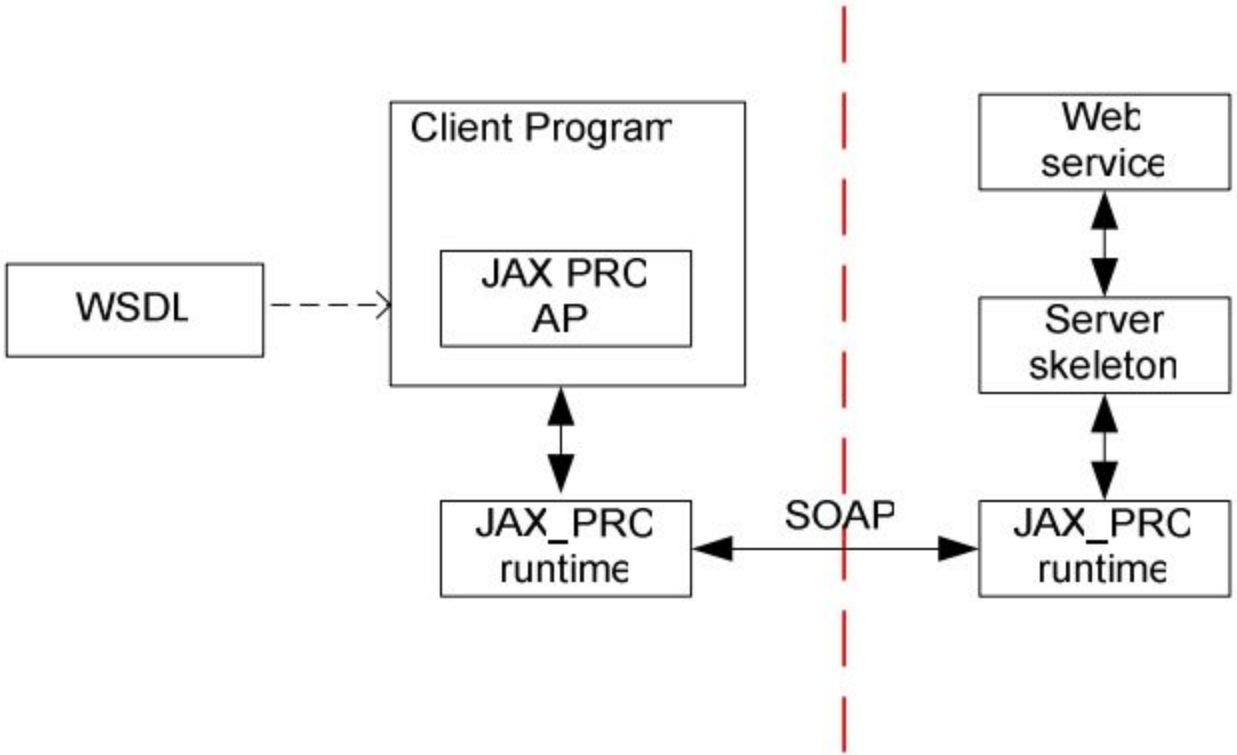


Figure 4-2- 4 Dynamic invocation interface model

in the static stub based method, both service interface and service implementation (static stub) are created at compile time; As to the dynamic proxy, only the service interface is created at compile time while implementation (dynamic proxy) is generated at runtime; Lastly, the DII method produces both service interface and implement at the runtime.

Another good example to explain the differences between above 3 types of communication models is to compare the client implementation code. Let us say that there is a “say hello” Web service, which basically receives the user name, and print out the “Hello” with user name string. And we also are able to obtain its WSDL file

By means of static stub, the client first uses the JAX-RPC runtime toolkit (ex. Axis) to generate the client side static stub, and then easily program the client code to invoke the service. Figure 4-2-6 is a sample client program with using the static stub created by Axis. In Axis, a client program would not instantiate a stub directly. It would instead instantiate a server locator (HelloServiceLocator) and calls a get method which returns a stub (Hello)

```
<?xml version="1.0" encoding="UTF-8"?>
<wsdl:definitions
                    . . .
   <wsdl:message name="sayHelloRequest">
         <wsdl:part name="name" type="xsd:string"/>
   </wsdl:message>
   <wsdl:message name="sayHelloResponse">
         <wsdl:part name="sayHelloReturn" type="xsd:string"/>
   </wsdl:message>
   <wsdl:portType name="Hello">
         <wsdl:operation name="sayHello" parameterOrder="name">
            <wsdl:input name="sayHelloRequest" message="impl:sayHelloRequest"/>
            <wsdl:output name="sayHelloResponse" message="impl:sayHelloResponse"/>
         </wsdl:operation>
    </wsdl:portType>
    <wsdl:binding> . . .  </wsdl:binding>
    <wsdl:service name="HelloService">
         <wsdl:port name="Hello" binding="impl:HelloSoapBinding">
            <wsdlsoap:address location="http://localhost:8080/axis/services/Hello"/>
         </wsdl:port>
     </wsdl:service>
</wsdl:definitions>
```

Figure 4-2- 5 "say hello" Web service WSDL file

```
Import Hello_pkg.Hello;
import Hello_pkg.HelloServiceLocator;
public class helloClient {
     public static void main(String[] args) {
       HelloServiceLocator lo= new HelloServiceLocator();
       try{
          Hello svc = lo.getHello();
          System.out.println(svc.sayHello("myWorld"));
       }catch (javax.xml.rpc.ServiceException se){
                    se.printStackTrace();
                    return;
       }catch (java.rmi.RemoteException re){
                    re.printStackTrace();
                    return;     }
     } }
```

Figure 4-2- 6 Client code by using static stub

On the contrary, a dynamic proxy class can be use to create client invocation without requiring pre-generation of the implementation-specific proxy class, which is the case in the static stub communication model. It calls a remote procedure through a Import Hello_pkg.Hello; import Hello_pkg.HelloServiceLocator; public class helloClient { public static void main(String[] args) { HelloServiceLocator lo= new HelloServiceLocator(); try{ Hello svc = lo.getHello(); System.out.println(svc.sayHello("myWorld")); }catch (javax.xml.rpc.ServiceException se){ se.printStackTrace(); return; }catch (java.rmi.RemoteException re){ re.printStackTrace(); return; } } } - 36 - dynamic proxy, a class that is created during runtime. Figure 4-2-7 shows how the client code might use a dynamic proxy instead of a static stub to access a service. First, it creates a Service object named ots from the createService method by the ServiceFactory object. Secondly, the client code creates a proxy (dyproxy) with a type of the service endpoint interface (Hello_pkg.Hello), which is generated by Axis tool

```
import javax.xml.rpc.ServiceFactory;
import java.net.URL;
import javax.xml.rpc.Service;
import javax.xml.namespace.QName;
public class helloClientDy {
     public static void main(String[] args) {
          try{
          ServiceFactory sf = ServiceFactory.newInstance();
          String wsdlURI =
               "http://localhost:80/axis/services/Hello?WSDL";
          URL wsdlURL = new URL(wsdlURI);
          Service ots = sf.createService(wsdlURL,new QName("urn:Hello", "HelloService"));
          Hello_pkg.Hello dyproxy = (Hello_pkg.Hello)ots.getPort(new QName("urn:Hello",
"Hello"), Hello_pkg.Hello.class);
          System.out.println(dyproxy.sayHello("myWorld"));
          }catch (Exception e){
               e.printStackTrace();
               return;
          }    }}
```

Figure 4-2- 7 Client code by using dynamic proxy

As come to the DII communication model, the client code is more complex than others by the reason that the code solely depends on the WSDL file. The DII client does not require runtime classes generated by any toolkit, like Axis, or wscompile. Instead, the programmer needs to specify the operation name, operation parameters and return type. All the information is obtained from the WSDL file. Figure 4-2-8 illustrates a piece of client DII code.

the kSOAP does not provide any support to generate static stub and service interface. The developer has to use the DII method to invoke the service on the server side. The good aspect to use DII is that the developers have complete control to client programmer; however the programming complexity is also raised. Typically, the client needs to create Call object first and set the operation and parameters during runtime

```
import javax.xml.rpc.Call;
import javax.xml.rpc.Service;
import javax.xml.rpc.JAXRPCException;
import javax.xml.namespace.QName;
import javax.xml.rpc.ServiceFactory;
import javax.xml.rpc.ParameterMode;
public class helloClientDII {
   private static String BODY_NAMESPACE_VALUE =  "urn:Hello";
   private static String ENCODING_STYLE_PROPERTY =
       "javax.xml.rpc.encodingstyle.namespace.uri";
   private static String NS_XSD = "http://www.w3.org/2001/XMLSchema";
   private static String URI_ENCODING = "http://schemas.xmlsoap.org/soap/encoding/";
   public static void main(String[] args) {
      try {
         ServiceFactory factory = ServiceFactory.newInstance();
         Service service = factory.createService( new QName("HelloService"));
         Call call = service.createCall(new QName("Hello"));
         call.setTargetEndpointAddress("http://localhost:80/axis/services/Hello");
         call.setProperty(Call.SOAPACTION_USE_PROPERTY, new Boolean(true));
         call.setProperty(Call.SOAPACTION_URI_PROPERTY, "");
         call.setProperty(ENCODING_STYLE_PROPERTY, URI_ENCODING);
         QName QNAME_TYPE_STRING = new QName(NS_XSD, "string");
         call.setReturnType(QNAME_TYPE_STRING);
         call.setOperationName(new QName(BODY_NAMESPACE_VALUE,"sayHello"));
         call.addParameter("String_1", QNAME_TYPE_STRING, ParameterMode.IN);
         String[] params = { "myWorld!" };
         String result = (String)call.invoke(params);
         System.out.println(result);
      } catch (Exception ex) {
```

Formulate a Call Once a service is located, the client needs to formulate a call to invoke the service. If this is the case, the methods _setProperty and _getProperty defined by the javax.xml.rpc.Call are particularly useful. Figure 4-2-12 client code by using DII illustrates setting the properties on the Call interface. Besides, the mapping between SOAP-defined types used by the service and the Java-defined types used by the client application is also an important issue when - 38 - formulate a service call. As we all understand, when a client invokes a service, the JAX-RPC runtime maps java type parameters to the corresponding SOAP representations and sends a SOAP message via HTTP request to the service. Once the service responds to the request, the JAX-RPC runtime need to map the SOAP type return values back to Java objects. When stubs are used, take Apache Axis for example, the tool WSDL2Java maps parameters, exception and return values type from xml type into the generated classes.

Process the Return Values As we see above, once the client invoke the call and the service responds to the request, the JAX-RPC runtime needs to map the SOAP type return values back to Java objects. In the most cases, the client might like to display the result in a Web page using a HTML browser. The J2EE platform provides a rich set of component technologies to support such demands, such as JavaServer Pages (JSP) technology. However, as we are only going to build a demo J2ME application as Web service consumer in the thesis project, for the demonstration purpose, displaying the return values as Java object to the client application is already good enough.

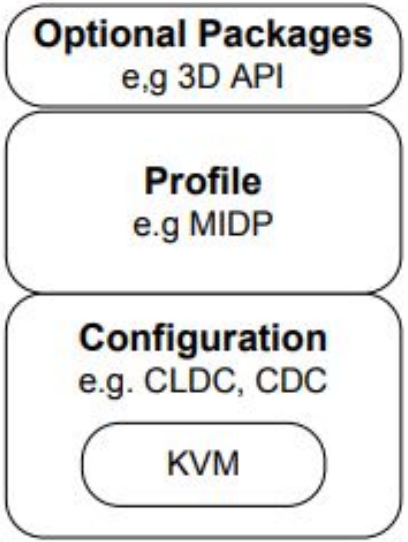
What is J2ME? Briefly, J2ME combines a resource constrained JVM and a set of Java APIs for developing applications for mobile devices.[24] Contrast to a traditional Java Virtual Machine (JVM), the JVM running on micro device (actually called KVM in the case) is very limited and supports only a small number of traditional Java classes. Usually, the device manufacturers will install and prepackage the device with this KVM and associated APIs. The developers only takes care the limited functional APIs and program applications targeting the device.

J2ME can be divided into three layers

Configuration: contains KVM and some class libraries. The most popular configuration that Sun provides is Connected Limited Device Configuration (CLDC). CLDC is for device with very limited configuration, like 16/32bit microprocessor and 160kb-512kb available memory. For the more feature-rich device with at least 2 MB of memory available, the configuration is correspondingly Connected Device Configuration (CDC). Profile: Profile also contains a set of API which is defined for a specific series of device and it is implemented above certain configuration layer. The developers just develop the applications within some specific profile. For the connected limited device like: PDA and cell phone, their profile layer is called Mobile Information Device Profile (MIDP). MIDP consist of a series of APIs that allow the developers to create not only the customized GUI shown on a mobile device but also full-scale business applications using external and internal data sources. Optional packages: an optional set of APIs that may or may not contained in the application, like media API and 3D API.

Mobile Information Device Profile (MIDP)

All applications for the MID Profile must be derived from a special class, MIDlet. The MIDlet can be compared to J2SE applets, which is a small program running on mobile device. A MIDlet can exist in four different states: loaded, active, paused, and destroyed.



re 5-3- 1 The J2ME stack [24

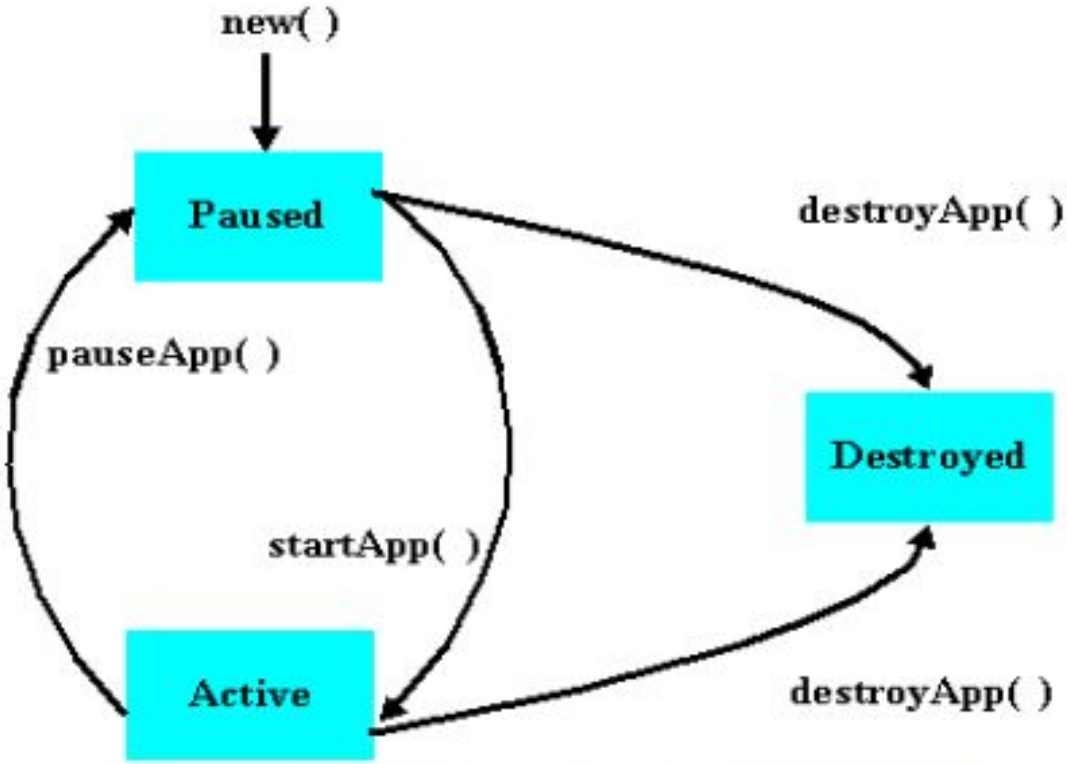


Figure 5-3- 2 Life cycle of a MIDlet [48]

The implementation of MIDlet is manipulated by Application Management software (AMS) which is also called Java Application Manager (JAM) and actually responsible for the whole function mechanism of MIDlet. JAM can destroy or turn the MIDlet to paused status and activate it again based on these three MIDlet states. Every time when JAM try to create a new instance of MIDlet, it will call the constructor of MIDlet first and let it be in the paused status and then make MIDlet enter into Active status by calling the method MIDlet.startApp(). After this JAM can either make MIDlet back to paused status again by calling MIDlet.pauseAapp() or destroy the MIDlet by calling the MIDlet.destroyApp().From a programmer's point of view, the state of MIDlet can be changed by calling methods: resumeRequest, notifyPaused and notifyDestroyed.

SDk Example

Sony Ericsson SDK 2.2.4 for the Java ME platform [52] It is a customized version of Sun Microsystems' J2ME Wireless Toolkit version 2.2 which includes a device customization for the Sony Ericsson MIDP2 mobile phones. The reason why we choose Sony Ericsson is we have a Sony Ericsson handset K700 on which we can test our MIDlet in a real environment. And also the Sony Ericsson J2ME SDK can be integrated with IDE, like, Eclipse. The SDK supports Universal Emulator Interface (UEI) and can be integrated with Eclipse that also supports UEI. The required software includes Java SDK1.4.1 or higher and DirectX8.1 or later.

Test As part of the wireless toolkit, we use an emulator device that mimics the functionality of a real mobile device via PC to test our implemented MIDlet. To run the emulator, hit the Run menu, an emulating k700 mobile pop up on the screen

Figure 5-3- 9 Test the MIDlet project (1/3)

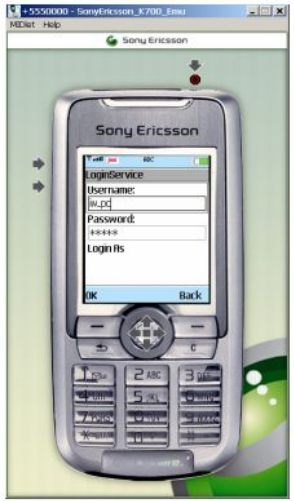


Figure 5-3- 10 Test the MIDlet project (2/3)

Figure 5-3- 11 Test the MIDlet project (3/3)

Deploy After testing, we only verified that our MIDlet was running OK on the simulated environment. But how is about on the real device? How to deploy the MIDlet directly on it? For the J2ME enabled phone, there are two options available. The first is via a network connection between the computer and the handset, like, via USB cable, via Bluetooth wireless connection. Second, we can also deploy the MIDlet by using Internet connection, provided the device is able to access the Internet using it internal Web browser. The basic idea is to put the MIDlet JAD file on the Internet, and let the mobile device find, download, install and run it. Usually, the developer will create a HTML file which has a link to the MIDlet JAD file, and the JAD file provides a link to the associate JAR file via the MIDlet-Jar-URL: THclint.jar attribute. Upload the newly created HTML file, the JAD file, the original JAR file to the web server and make it available to the mobile device browser, so anyone with a mobile device that can browse the Internet should be able to point to the deployed MIDlet

If you wish you can Read the documentation of Docker and Kubernetes.